



Brandenburg
University of Technology
Cottbus - Senftenberg

Comparative Analysis of Anomaly Detection in Ultrasonic Wireless Sensor Networks, using BigQuery ML

Master Thesis

Seyed Ehsan Hashemi Rastegar

Date of Submission

28 November 2022

This report is submitted to the

Chair of Wireless Systems

Brandenburg University of Technology, Germany

Supervisors:

Prof. Dr. Peter Langendörfer

Dr. Svetlana Meissner

Mentors and Examiners from BTU:

Prof. Dr. Peter Langendörfer

Dr. Svetlana Meissner

Contents

List of Figures	iii
List of Tables	v
Acknowledgment	1
Abstract	3
1 Introduction	5
2 Embedded systems	7
2.1 Overview [29]	7
2.2 Challenges [29]	7
2.2.1 Dependability	8
2.2.2 Real-time constraints	9
2.2.3 Efficiency	9
2.2.4 Adaptation of big data and machine learning problems	10
2.3 Conclusion	11
3 Data Warehouse as a Service (DWaaS)	13
3.1 Introduction	13
3.2 Overview [18]	13
3.3 Data Warehouse as a Service [18] [7]	13
3.4 Components of a DWaaS [18]	14
3.5 BigQuery ML [6] [7]	14
3.6 Supported Models in BigQuery ML	15
3.7 Conclusion	16
4 Anomaly Detection	17
4.1 Introduction	17
4.2 Anomaly Detection	17
4.3 Types of Anomalies [11]	17
4.3.1 Point Anomaly	18
4.3.2 Contextual Anomaly	18
4.3.3 Collective Anomaly	18
4.4 Anomaly Detection Methods [34]	18
4.4.1 Statistical Model-Based Anomaly Detection Approach	19
4.4.2 Density-Based Anomaly Detection Approach	19
4.4.3 Distance-Based Anomaly Detection Approach	19
4.4.4 Clustering-Based Anomaly Detection Approach	19
4.4.5 Isolation-Based Anomaly Detection Approach	20
4.4.6 Ensemble-Based Anomaly Detection Approach [40]	20
4.4.7 Subspace-Based Anomaly Detection Approach	20
4.5 Anomaly Detection methods in the Fraud Detection domain	22
4.5.1 Detecting anomalies based on the statistical approach	22

4.5.2	Detecting anomalies based on the machine learning approach	22
4.5.3	Detecting anomalies based on the deep learning approach	23
4.6	Conclusion	23
5	Anomaly Detection of Wireless Sensor Network data using Machine Learning Techniques	25
5.1	Introduction	25
5.2	Labelled data in Anomaly Detection techniques	25
5.2.1	Supervised Learning for Anomaly Detection	26
5.2.2	Semi supervised Anomaly Detection	26
5.2.3	Unsupervised Anomaly Detection	26
5.3	Use case of this study	27
5.3.1	Feature data and types of sensors	27
5.4	Detailed view of the research design	29
5.4.1	Anomaly Detection in Fraud Detection Analysis	30
5.4.2	Anomaly Detection in Time Series Analysis	31
5.5	Data Analysis	32
5.5.1	Data analysis of processed data	32
6	Data Modeling and Results	37
6.1	Introduction	37
6.2	Data Models	37
6.2.1	Fraud Detection Data Models	38
6.2.2	Handling the Imbalanced dataset [25]	41
6.2.3	Time Series Data Models	48
7	Conclusion	55
8	Appendix 1: Autoencoder	57
9	Appendix 2: Imbalanced learning	59
10	Appendix 3: Logistic Regression	61
11	Appendix 4: Models evaluations	65
12	Appendix 5: Time Series Analysis	69
	Abbreviations and Acronyms	71
	Bibliography	73

List of Figures

3.1	BigQuery Service Model [7]	15
4.1	Autoencoder Network [23]	23
5.1	Choosing the right Anomaly Detection technique based on a given amount of labeled data available [23]	26
5.2	Anomaly Detection Pipeline	30
5.3	Ultrasonic Level Diagram	33
5.4	Aggregated Ullage Measured Values - KFA5-17 Sensor Device	34
5.5	Aggregated Ullage Measured Values - KFA5-16 Sensor Device	34
5.6	Temperature Measured Values - KFA5-17 Sensor Device	35
5.7	Temperature Measured Values - KFA5-16 Sensor Device	35
5.8	Temperature Measured Values - KFA5-18 Sensor Device	36
6.1	Reconstruction Loss - Autoencoder model	40
6.2	Autoencoder reconstruction losses trends	41
6.3	Feature Importance - Logistic Regression models (model_01, model_02)	44
6.4	Evaluated Metrics - Logistic Regression models (model_01, model_02)	45
6.5	Evaluated performance curves - Logistic Regression models (model_01, model_02)	46
6.6	Time series analysis workflow (daily transactions) - KFA5 Sensor Devices	49
6.7	Time series analysis workflow (hourly transactions) - KFA5 Sensor Devices	50
6.8	Anomalies of total daily transactions (KFA5 All Sensor Devices)	51
6.9	Anomalies of total daily transactions (KFA5 Active Sensor Devices)	52
6.10	Anomalies of total hourly transactions (KFA5 All Sensor Devices)	53
6.11	Anomalies of total hourly transactions (KFA5 Active Sensor Devices)	54

List of Tables

3.1	Supported models in BigQuery ML [43]	16
4.1	Comparison of anomaly detection techniques [34]	21
5.1	Datasets of Proteus level sensors	27
5.2	KFA5 dataset properties	28
5.3	KFA5 Active Sensors	29
5.4	Processed data schema for model creation	32
6.1	Data modeling pipeline	37
6.2	Autoencoder parameters	39
6.3	BigQuery ML reconstruction losses output function (Autoencoder)	39
6.4	Autoencoder prediction results	41
6.5	Confusion Matrix (model_01, model_02)	47
6.6	Evaluated metrics - test data records	47
6.7	Anomalies of total daily transactions (KFA5 All Sensor Devices)	51
6.8	Anomalies of total daily transactions (KFA5 Active Sensor Devices)	52
6.9	Anomalies of total hourly transactions (KFA5 All Sensor Devices)	53
6.10	Anomalies of total hourly transactions (KFA5 Active Sensor Devices)	54
12.1	Abbreviations and Acronym	71

Acknowledgment

First and foremost, I would like to express my genuine appreciation to my advisors, Professor Dr. Peter Langendörfer and Dr. Svetlana Meissner, for their endless support and guidance throughout the process of this MSc. thesis research. I'm grateful for this opportunity to work on the topic of Anomaly Detection through the lens of security and machine learning. Indeed, it was a great learning curve.

Last but not least, my profound appreciation goes to my family: my parents, and my brothers for their countless support and continuous encouragement. Thank you.

Ehsan

Abstract

Author: Seyed Ehsan Hashemi Rastegar

Topic: Comparative Analysis of Anomaly Detection in Ultrasonic Wireless Sensor Networks, using BigQuery ML

Chair of Wireless Systems, Brandenburg University of Technology, Germany, 2022, p 75

Supervisors: Prof. Dr. Peter Langendörfer, Dr. Svetlana Meissner

Wireless Sensor Network is an infrastructure-less wireless network aggregated from multiple sensors that monitor the system or environmental conditions. The cost-effectiveness and efficiency to be adapted in various applications like the Internet of Things (IoT), security monitoring, and threat detection gained traction over recent years. Such attention raises security vulnerabilities that must be protected against security attacks, information leakage, and tampering. However, adding advanced security mechanisms to the embedded systems is not ideal for manufacturers due to the constraints surrounding computing power, battery lifespan, manufacturing cost, and regularity standards compliance. On the contrary, in recent years, cloud computing has been demonstrated as a compelling solution to address embedded systems' limitations and move forward. Data Warehouse as a Service (DWaaS) considers one of the recent cloud computing services that can perform such data processing on the cloud. Google's BigQuery ML is an extension of google's DWaaS platform that supports machine learning techniques. Anomaly detection is a process that detects abnormal system behaviors that deviate from normal behaviors. This research studies the anomaly detection techniques of Proteus-level sensors in two compartments: fraud detection and time-series analysis. The results from the fraud detection analysis show that based on the processed dataset, in imbalanced learning, having a weighting feature enables a higher model performance compared to the under-sampling technique.

Keywords: Wireless sensor networks, DWaaS, Data Warehouse as a Service, anomaly detection, semi-supervised learning, supervised learning, BigQuery ML, fraud detection analysis, time series analysis

1 Introduction

With the enhancement of the Internet of Things (IoT) in the recent era, the real-world applications of Wireless Sensor Networks (WSNs) are becoming increasingly popular in the computer network industry. With this growth and recognition, additional security measures should be enforced to protect the data traffic between WSNs from outsider attacks. However, adding more advanced cryptographic mechanisms on the hardware to secure the communication would not be ideal for embedded systems due to their constraints surrounding computing power resources, battery lifespan, and cost. In other words, manufacturers must upgrade the microcontrollers to fulfill this need which can increase the overall cost. Furthermore, over the past few years, cloud computing elevated in the market tremendously to shift the load of processing burden from embedded systems to the cloud. In particular, always having active sessions between WSN nodes to transmit the measured data (traffic) will increase energy consumption by generating secure handshakes and being on the active mode status.

On the contrary, it is possible to shift the load of such security validation and data analytical tasks to the cloud. Which is scalable in terms of computing resources and storage, provides more flexibility to manage the software and hardware¹ (infrastructure in the cloud); and reduces overall manufacturing costs. Furthermore, by doing so, less complex and inexpensive micro-controllers must be deployed on these WSN nodes to operate. At the same time, the measured data can be transmitted in batches to the cloud to improve energy utilization on the device.

Data Warehouse as a Service (DWaaS) is a suitable cloud computing service that supports such demand by collecting and processing data on the cloud. Google BigQuery ML is a SQL syntax-based DWaaS platform that performs machine learning (ML) techniques, including security validations and data analytical tasks of the data stored in the BigQuery ecosystem. Anomaly Detection (AD) is a machine learning process that detects abnormal system behaviors that deviate from normal behaviors. With the help of AD techniques, the traffics sent from sensor nodes can be observed, and any malicious observations can be reported for further investigation.

The motivation behind this research is to conduct the security analysis using Anomaly Detection techniques based on the real Big Data obtained from WSN node transaction records on the BigQuery ML. The analysis covers two compartments: (1) Anomaly Detection in Fraud Detection Analysis and (2) Anomaly Detection in Time Series Analysis. The fraud detection approach uses a deep learning method (Autoencoder) to detect anomalous records. Then the results (training data) are incorporated to train two classification models using the Logistic Regression method. The study showed that model_01, with an imbalanced dataset and *auto_class_weights* function enabled, provides an accurate prediction for the test data compared to model_02, which trains with the under-sampling technique. Furthermore, in time series analysis, the ARIMA_PLUS method is used to identify anomalies based on the total number of transactions, grouped by daily and hourly timeframes, for two categories of datasets: (1) Active and non-active sensors and (2) Active sensors. Four models are compiled in this compartment.

This research is divided into six chapters: Chapter 2 covers embedded systems alongside with its challenges during the design process, in chapter 3, DWaaS is introduced, along with its benefits,

¹due to some standards regulatory constraints, upgrading and modifying the embedded systems is more rigid in contrast to the cloud

and components, followed by the introduction to BigQuery ML and its capability to detect anomalies (in particular towards anomaly detection approach). Chapter 4 discusses the concept of anomaly detection, types of anomalies, AD techniques, and the comparison between these techniques, followed by the AD methods in the fraud detection domain. In chapter 5, labeled data in anomaly detection is introduced, followed by the use case of this study, a detailed view of the research design, and the data analysis of processed data are discussed. In chapter 6, data modeling is introduced, followed by the seven models² compiled for anomalies in fraud detection, and time series compartments are explained. Lastly, in chapter 7, the conclusion of this research is outlined.

²the first model is compiled based on the Autoencoder (deep learning) method, the second and third models are based on the logistic regression (classification) technique, and the last four models are compiled based upon the ARIMA_PLUS approach for time series analysis

2 Embedded systems

This chapter presents the overview of embedded systems, the history of miniaturization in information processing, and the integration of embedded systems with the Internet of Things (IoT) in section 2.1. The imposed challenges of embedded systems during the design process covers in section 2.2. Lastly, in section 2.3, cloud computing is introduced as a suitable solution to overcome these challenges in connection with big data and machine learning problems.

2.1 Overview [29]

The history of miniaturization¹ began in the 1980s when information processing was linked with large mainframe computers, and later this capability to process information was brought to personal computers (PCs). Mark Weiser had this vision that computers would be integrated into products, and such a trend could lead computers to be part of our day-to-day life for computing processes. The essence behind "pervasive computing" focuses more on the practical attributes of computing and the integration with the available technology. Moving forward to the recent developments of miniaturization, where the integration of information processing with environmental sensing using computers took place, the term "embedded system" was introduced. According to Marwedel (2021), embedded systems are "information processing systems embedded into enclosing products." (Marwedel, 2021, p.2)

Integrating embedded systems with the Internet of Things (IoT) facilitates industries to take advantage by applying them in day-to-day tasks. Such adaptations can observe in industries like automobiles (self-driving cars), healthcare (health and monitoring), infrastructure (CCTV²), agriculture, retail, fitness, and many more. The term Internet of Things "*describes the pervasive presence of a variety of devices - such as sensors, actuators, and mobile phones - which, through unique addressing schemes, are able to interact and cooperate with each other to reach common goals.*" [14]. Within the IoT context, sensors provision the sensed information to be attainable on the Internet, and actuators control such data from the Internet [29].

Although embedded systems in IoT bring immense benefits to the table, some considerations must be addressed, especially concerning security by design. The next section discusses the common characteristics of embedded systems alongside the challenges.

2.2 Challenges [29]

According to Marwedel [29], there are three common characteristics in embedded systems that must be addressed during the design phase: (1) dependability, (2) real-time constraints, and (3) efficiency. In addition, in the later part of this section, the challenges of adapting embedded systems in IoT and, more specifically, those problems related to big data and machine learning are discussed.

¹Miniaturization refers to the tendency to manufacture smaller mechanical and electronic products and devices.

²CCTV stands for Closed Circuit Television.

The three characteristics of embedded systems comprised of the following parameters:

2.2.1 Dependability

A system considers dependable when it can offer its deliberated service with a high probability without causing any destruction [29]. That is to say, deploying these systems in the environment directly impacts the environment. There are six main constituents involved in achieving a highly dependable system:

1. Security

The core principle of information to be secured is based on the CIA³ triad. With the enhancement of embedded systems in IoT, security plays a crucial role in providing such assurance, especially since IoT requires more components to be connected. That opens a window of opportunity for attackers if proper security measures are not in place. Kocher et al. [20] outline such security measures in five domains:

To initiate a secure connection, firstly, an appropriate validation mechanism must transpire to ensure the identities of a user/actor are validated prior to granting access. This stipulation is often known as **identification and authentication**. The second measure occurs during the **authorization** phase, where only authorized device is allowed to utilize a service or network connection. The third measure emphasizes **securing the communications** by including appropriate communication features. The fourth area focuses on **securing the storage** of information which can guarantee the confidentiality and integrity of data. Lastly, **content security** is another vital aspect of information security that induces restricting usage.

2. Confidentiality

Confidentiality ensures that the information is not accessible by unauthorized individuals or resources. Encryption enables security for data confidentiality. However, having advanced encryption mechanisms in embedded systems impacts a system's overall performance and energy consumption.

3. Safety

Safety is an important aspect that must be considered during the design process. It ensures people's health, either directly or indirectly, is guaranteed in case of any damage to the property or the environment. Functional safety ensures that a system operates according to the expected and deliberated functions. In computer systems, safety refers to the risks corresponding to failures that do not occur by external factors like hardware or power failures or any operational errors and misbehavior.

4. Reliability

A malfunctioning system refers to a situation when components of the system are not functioning as per the intended design time. Reliability is the probability of a system not having malfunctioning behavior within an explicit time frame. Reliability is mainly measured based on the impacts inside the system within the intended regular operation; any external factors like malicious attacks do not fall under this consideration.

³CIA stands for confidentiality, integrity, and availability

5. Repairability

Repairability is measured based on the likelihood that a system can be repaired within the allocated time frame in the event of such failure.

6. Availability

The prerequisite of having a highly available system comes with a design that ensures a system's high reliability and repairability are fulfilled.

2.2.2 Real-time constraints

Real-time constraints are another characteristic of embedded systems. If the computation cannot be completed within the intended time frame, it impacts the overall quality of the service being delivered. Although the real-time constraints are subjective to the specific applications of embedded systems, some applications are reactive; in vehicle engineering, if the Anti-Lock Braking System (ABS) is not operating in time, that could affect the safety of passengers. These constraints can generally be categorized into *hard* or *soft* time constraints. In hard time constraints, such failure could result in a catastrophic situation.

2.2.3 Efficiency

Efficient resource utilization is the third characteristic of embedded systems. Marwedel [29] describes five metrics that are essential for evaluating the resource efficiency in embedded systems:

1. Energy

“Electronic information and communication technology (ICT) uses electrical energy for information processing and communication” [29].

This electrical energy converts to thermal energy, which can then be used for operating embedded systems. Electrical power and energy are vital constraints in embedded systems. Having a higher computing performance results in increased energy consumption. If the energy cost is relatively expensive, it directly impacts the computational cost for a system to be operational. Furthermore, thermal effects are another limitation in the context of energy in embedded systems. The increased temperature on the circuits has a direct impact on the overall reliability of the circuits. In other words, the more energy consumed during a system's computing process, the reliability of a system drops over time. Therefore, in unplugged systems, it is crucial to ensure that energy-saving consideration is in place due to the limited capacity of batteries in embedded systems.

2. Run-time

If a system is designed with counterproductive execution time and a considerable amount of processor cycles are being wasted, that impacts the system's overall efficiency.

3. Code size

In some embedded systems like systems on a chip (SoCs), there is a constraint with regard to the storage capacity in which, the code needs to be stored on the system. In order to have a optimum execution on the memory, the code requires to be compact.

4. Weight

A lightweight, portable embedded system is another aspect that can influence overall efficiency. This metric is applicable mainly for embedded systems that are on the move. For example, in the transportation and mobility domain, e-scooters are a prime example that requires sensors (embedded systems) to be attached to the vehicle to trace their locations on the Internet. E-scooter providers use these data for collection at the end of the day for repair, battery replacement, or even allocating them to more strategic locations. The weight of the embedded system on these e-scooters directly impacts the overall battery durability.

5. Cost

The manufacturing cost of embedded systems in the consumer electronics market can be broken down into a few areas. Yet, the overall goal for the manufacturers is to offer a competitive and affordable price for consumers. These considerations could be related to the allocated budgets for software development stages, energy-efficient hardware components with minimum resource utilization usage, or reducing supply voltages and clock frequencies [29].

That is to say, to come up with an efficient manufacturing cost for embedded systems, there are many factors involved throughout the whole design and construction processes.

The above metrics highlight the importance of an efficient design process for embedded systems that are correlated to one another. Looking at a higher glance shows that software and hardware designs must rely on each other to provide the optimum output results for the system. However, hardware implementation comes with some limitations. Such changes require lengthy design and approval processes, leading to the notion that hardware implementations are costly. Moreover, due to this rigidity⁴ of hardware implementations, it is not ideal to obtain efficiency through hardware. The compromise should focus on the other alternative solution that can offer flexibility while it is efficient.

2.2.4 Adaptation of big data and machine learning problems

By nature, embedded systems in IoT require collecting large amounts of data for processing and analysis. However, due to the limitations mentioned above, if a system desires to perform computationally expensive data processing (e.g., using big data with the help of machine learning techniques for real-time analysis) on-premise, a large storage volume, and advanced micro-controllers are needed for provisioning. However, this requisite eventually surges the components' procurement costs and the overall manufacturing budget.

Furthermore, performing the computation and processing on-premise is not optimum for resource utilization and resource efficiency, which impacts the battery lifespan and overall reliability over time.

As an alternative, while the solution needs to ensure that all the characteristics of embedded systems are fulfilled, the loads for computational and storage burdens can be shifted to cloud computing technology. The cloud solution embraces the following advantages:

First and foremost, less complex components need to be purchased, which then reduces the hardware costs. Next, fewer hardware changes or implementations are required. Moreover, cloud computing indulges a green environment and protects from global warming. In terms of technical aspects, processing big data on the cloud and performing machine learning-related analysis

⁴not being flexible

becomes more reliable, flexible, and efficient in a highly scalable environment. In other words, less reliability on a system increases the chance of the device or components being replaced more frequently, directly impacting the environment in connection with a sustainable life cycle.

Concerning security, cloud solution offers adequate visibility and transparency for system administrators to react to suspicious transactions in real-time compared to the conventional approach, where embedded systems operate in a decentralized network. Furthermore, having a centralized management means is substantially beneficial when dealing with large amounts of data that need to be processed and analyzed using an intelligent apparatus.

2.3 Conclusion

This chapter discusses the history of embedded systems, their integration with IoT, and their characteristics. These characteristics pose unique and exciting challenges. *Embedded system security* is a vital aspect that must be considered during the design process while construing other metrics like performance, power efficiency, and overall cost. Adaptation of new technologies (e.g., big data and machine learning) raises challenges for embedded systems that require a flexible, scalable, and effective solution. Cloud computing offers such benefits while devoting security. The next chapter discusses cloud computing and, in particular, one type of its service model (Data Warehouse as a Service).

3 Data Warehouse as a Service (DWaaS)

3.1 Introduction

This chapter covers the overview of cloud computing solutions in section 3.2. Data Warehouse as a Service introduces in section 3.3 along with its benefits. The components of DWaaS discusses in section 3.4. Followed by the introduction to BigQuery ML, its advantages (section 3.5), and the supported models for the associated use cases¹ (section 3.6). Lastly, the chapter concludes in section 3.7.

3.2 Overview [18]

In recent years, software providers have been shifting towards the cloud, as cloud solution provides more attractive benefits than the conventional computing infrastructure setup. This shift leads toward delivering software as a service. Cloud computing is scalable, agile, and cost-effective. Cloud computing can be viewed from two aspects: (1) Based on the resources being used/utilized; and (2) Based on the services being offered.

From the lens of resource utilization, there are three: public, private, and hybrid cloud models. *Public* cloud refers to when resources such as applications and storage are publicly available and are being shared among organizations. The organization has minimum visibility and control over the computing infrastructure in the public cloud, and the data is being processed. In *Private* cloud, dedicated computing infrastructure assigns to each organization, providing higher data security but at a higher cost. Finally, in *Hybrid* model, an organization uses its computing infrastructure and the cloud during high demands. It is a hybrid version of the first two models.

From the lens of service offerings, cloud models are classified into multiple types: IaaS (Infrastructure as a Service), PaaS (Platform as a Service), Serverless, FaaS (Function as a Service), SaaS (Software as a Service), and DWaaS (Data Warehouse as a Service). For example, figure 3.1 shows the control level for an organization over different types of cloud services.

3.3 Data Warehouse as a Service [18] [7]

In a conventional approach of data warehouse solution, companies must host the data warehouse model on-premise, which could raise additional efforts on maintaining the hardware to be provisioned, purchased, deployed, and even patched with the latest security updates. Therefore, shifting the data warehouse to a cloud model helps an organization to reduce the workforce resources and the responsibility for maintaining the computing resources (servers, storage, and networking) [18] [7].

By leveraging DWaaS, the following benefits can lead to a cost reduction for an organization: Firstly, the "pay-as-you-go" model enables businesses to pay as per usage only (per request). That helps to focus on the operational cost for the companies. Secondly, the hardware resources

¹in particular, this research study focuses on the concept of anomaly detection

can be maximized to utilize these resources effectively. Thirdly, DWaaS is a highly scalable service; more specifically, the organization can scale up the storage resources without interrupting the service. Furthermore, DWaaS provides flexibility for the user to manage how to utilize the software and hardware. Lastly, DWaaS enables companies to shift the IT operational tasks toward the automation approach, which can significantly help to reduce the manual workforce and increase productivity [18].

Regarding security, analysts or engineers only must focus on their applications, and the responsibility of maintaining the security updates for the software is shifted to the DWaaS provider [18]. In addition, the complexity of managing and maintaining the over/under provisioning will be shifted to the DWaaS provider as well [7].

3.4 Components of a DWaaS [18]

There are two main components in the DWaaS model: (1) ETL (Extract, Transform and Load) Cloud, and (2) BI Cloud.

In the ETL Cloud, three processes are involved in executing reports or queries from the database. Firstly, the data is extracted from an external source. Then it transforms based on the defined operational necessities, and lastly, it loads to a database (data warehouse). During the data transformation phase, ample amounts of I/O activity, string processing, and a transformation of variables, along with data parsing tasks, are being performed, which is compute-intensive [24]. As an alternative approach, ELT (Extract, Load, Transform) method shifts the load of computing the transformation activity from an on-premise server to the cloud, which is faster and more efficient [24].

The BI Cloud offers services related to reporting, such as business or financial reporting, forecasting, and budgeting. An organization can observe the network traffics more explicitly, and in general, Business Intelligence (BI) related activities can be more transparent for the organization, including the computing utilizations. Another nifty feature BI Cloud offers is the ability for businesses to set the threshold for resource utilization. By doing so, the organization has complete control over the usage in real-time to avoid exceeding the budget.

3.5 BigQuery ML [6] [7]

Google's BigQuery is a serverless data warehouse solution that has gained substantial popularity among organizations after its launch in 2011 [6]. BigQuery ML (Machine Learning) is an extension of BigQuery, a SQL syntax-based Data Warehouse as a Service (DWaaS) that enables the user to perform machine learning tasks in BigQuery as a PaaS solution.

According to [21], BigQuery can process the query from datasets with petabytes in size and "features data ingestion at the rate of 100,000 rows per second per table."

Some benefits of BigQuery ML are outlined: In BigQuery, functions can be a call or executed on demand. Resources like memory (RAM), processing power (CPU), and virtual instances can be provisioned without prior configuration. Analysts or engineers can focus on the query execution without concerning resource utilization (under/over provisioning), index configuration, backups, or any types of outages. In general, any related computing infrastructure of the service and data warehouse management topics, as all of these are being managed by the platform provider within the BigQuery ecosystem [7]. Another benefit is that if the requested query can be cached, there will not be any additional cost [7].

Figure 3.1 highlights the BigQuery service model where it fits in between PaaS and Serverless models [7]. The figure illustrates how the responsibility has shifted toward cloud providers in recent years as new cloud services are being introduced.

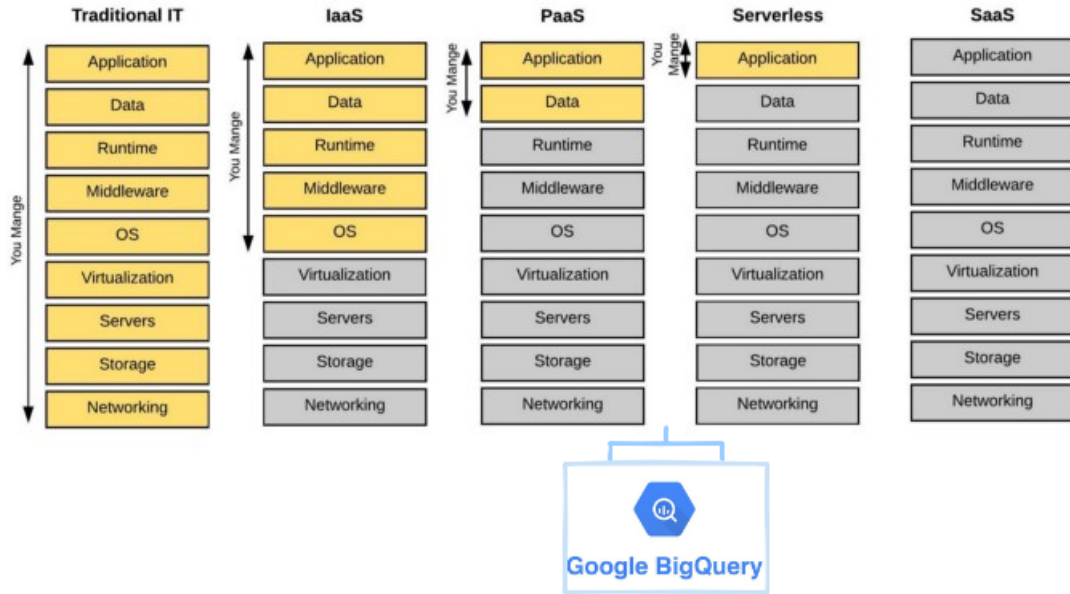


Figure 3.1: BigQuery Service Model [7]

3.6 Supported Models in BigQuery ML

BigQuery ML supports various models based on different tasks and use cases. Table 3.1 classifies the models into seven categories [43]. Throughout this research study, the anomaly detection domain is selected to delve into two compartments: (1) Fraud Detection Analysis, where the Autoencoder model is adopted, and (2) Time Series Analysis, where the ARIMA_PLUS model is used. In addition, Binary Logistic Regression (LOGREG) model is employed to classify normal and abnormal behaviors. In the next chapter, anomaly detection is discussed.

Tasks	Use cases	Models
Forecasting (predicting values)	Predicting sales figures Predicting stock prices	Linear Regression Boosted Trees Regressor AutoML Table Regressor DNN Regressor Wide & Deep Regressor
Predicting between categories	Identifying spam emails Identifying tumor types	Logistic Regression Boosted Trees Classifier AutoML Table Classifier DNN Classifier Wide & Deep Classifier
Generate Recommendations	Product recommendation	Wide & Deep Classifier Matrix Factorization
Reduce Data Dimensionality	Analysis of DNA data	PCA Autoencoder
Anomaly Detection	Fraud Detection Predicting credit risk	PCA Autoencoder K-Means ARIMA_PLUS
Clustering	Performing customer segmentation	K-Means
Time Series Forecasting	Predicting housing prices based on historical data	ARIMA_PLUS

Table 3.1: Supported models in BigQuery ML [43]

For more details, the following link (<https://cloud.google.com/bigquery-ml/docs/>) provides relevant documentation on how to set up and use BigQuery ML, along with training and tutorials are covered.

3.7 Conclusion

In this chapter, Cloud Computing, its service models, and DWaaS solution were discussed. BigQuery ML is a fully-managed serverless DWaaS that enables analysts or engineers to perform ML-related techniques at a large scale without managing the computing infrastructure bottlenecks. Lastly, the supported models in BigQuery ML were outlined. In the next chapter, the anomaly detection concept is discussed.

4 Anomaly Detection

4.1 Introduction

This chapter talks through the definition of anomaly detection (section 4.2), different types of anomalies (section 4.3). Followed by the techniques and methods of anomaly detection (section 4.4), and in particular the approaches of identifying anomalies in fraud detection domain (section 4.5). Lastly, the conclusion of this chapter covers in section 4.6.

4.2 Anomaly Detection

By defining a normal behavior of a system, any abnormal comportment that results in exceeding the expectation of having the acceptable conformable behavior correspond to these normalities is considered an anomaly or outlier [11]. The task of identifying such anomalies is known as anomaly detection.

Anomaly detection aims to address a machine learning or data mining problem [11]. The study of anomaly detection was initiated by [Edgeworth 1887] when statistical methods were proposed, and over the years, several anomaly detection techniques have been studied and developed by researchers. Chandola et al. (2016) expressed that one key distinctive characteristic of anomaly detection that distinguishes it from machine learning problems is that "anomaly detection is a highly application-oriented problem" (p. 2). Since defining anomalies are subject to the specific application domain, there is no solid ground to identify anomalies collectively [11]. For example, discovering anomalies in military surveillance or critical infrastructure for health monitoring systems could help identify early fault detection. In the cybersecurity domain, detecting anomalies of intrusion detection could help prevent the system from cyber-attacks. Hence, the techniques used for anomaly detection correlate with the specific application or use case that tends to be solved. Section 4.4 articulates anomaly detection techniques in detailed.

4.3 Types of Anomalies [11]

Depending on the type of anomalies, relevant anomaly detection techniques can be used to address the anomaly detection problem. This section discusses different types of anomalies, followed by section 4.4, to cover different types of anomaly detection techniques.

Generally, anomalies can be classified into three categories: (1) Point Anomalies, (2) Contextual Anomalies, and (3) Collective Anomalies. In *point anomaly*, a single point is considered anomalous irrespective of other anomalous points. On the contrary, a collection of data points in *collective anomalies* form an anomaly. In *contextual anomalies*, data points are considered anomalous when they are within a specific context.

4.3.1 Point Anomaly

*Point anomaly*¹ is the most common type of anomaly, whereas the intention is to identify the data points anomalies that deviate from normal behavior. In point anomaly detection, data points are instinctively isolated by which to determine the anomalous point. That is to say, in point anomalies detection, the anomalous points are observed individually to the normal data points. Another essential essence of point anomaly is that there is no other knowledge available to indicate if the point is within normal or abnormal behavior, which makes point anomaly detection an *unsupervised anomaly detection problem* (Chandola et al. 2016).

One of the most practical anomaly detection algorithms to detect point anomalies is the Local Outlier Factor (LOF). LOF algorithm tends to calculate the local deviation between a given point and the corresponding neighbors [27].

Figures 5.4, and figure 5.5 highlight the point anomalies of aggregated ullage values for KFA5-17, and KFA5-16 sensor devices. In addition, figure 5.6 highlights the point anomaly of temperature measured values for KFA5-17 sensor device.

4.3.2 Contextual Anomaly

In contextual anomaly (also known as *conditional anomaly*), within the given context, if the data points deviate from the rest of the data points, then they are considered contextual anomalies. For example, the sensor device indicates that a room temperature of 17 Celsius in winter could be normal. However, within the context that the sensor device is located outside the house, that is considered anomalous in winter. In other words, the context is part of the problem formulation to determine if the point is to be considered normal or abnormal.

Figure 5.8 highlights the contextual anomalies detected in KFA5-18 sensor device for temperature measured volume.

4.3.3 Collective Anomaly

In collective anomaly, the collection of affiliated data points in relation to the entire data set forms anomalies. Contrary to point anomalies, individual data points could be considered normal in collective anomalies, but collectively they could be anomalous.

Figures 5.4, and figure 5.5 illustrate the collective anomalies for the aggregated ullages of KFA5-17, and KFA5-16 sensor devices in blue.

4.4 Anomaly Detection Methods [34]

With the recent development of cloud computing and IoT, a wide range of application domains are leveraging anomaly detection techniques to improve the detection mechanism. This advancement in computing power made it possible for machine learning and deep learning technologies to be more effective in correlation with a large scale of data being collected from IoT devices [34]. Typically, anomaly detection techniques categorize into three main areas: (1) machine learning, (2) data mining, and (3) statistical techniques.

¹also known as global anomaly

In the *machine learning* approach, the system observes over time, and improvement will take place along the way. This improvement could be related to the performance or any other tasks that the machine could learn and enhance the process.

In the *data mining* approach, the patterns in data are analyzed, and any deviation or spike in the data raises the flag for anomalies. As Samariya et al. (2021) have mentioned, "anomaly detection is considered as one of vital task of data mining with wide range of application domains" (p. 2)

Lastly, in *statistical* approach, the detection mechanism compares the two-state profiles for each data object. If the comparison between the current and stored behaviors indicates significant disparity, then, that is to say, the point is anomalous.

Samariya et al. [34] classified the anomaly detection algorithms into seven domains: anomaly detection algorithms based on (1) statistic model, (2) density, (3) distance, (4) clustering, (5) isolation, (6) ensemble, and (7) subspace. At the end of this section, table 4.1 shows a comparison between these methods. The comparison articulates the pros and cons of each method.

4.4.1 Statistical Model-Based Anomaly Detection Approach

In the *statistical model* approach, initially, a statistical model based on the given dataset is formed. Then it will be evaluated whether the test data fit the statistical model. If it can be fitted, the data point is normal; otherwise, that would be an anomaly. In other words, outliers should reside in the model's low-probability regions. Based on the distribution of the given data during the modeling process, statistical models can be split into two branches: (1) *parametric algorithms* perform when the distribution is known², and in (2) *non-parametric algorithms* prior knowledge for distribution of data is not given.

4.4.2 Density-Based Anomaly Detection Approach

Density based anomaly detection algorithms compare the density of data points; if this disparity is significant for the data point to the normal instances, then that would be considered an outlier. LOF algorithm was designed and widely used based on this method.

4.4.3 Distance-Based Anomaly Detection Approach

In *distance* based approach, the distance between the data point and its k-nearest neighbors determines anomalies. If the sum value of this distance is significant, then that indicates an anomaly. One of the most commonly known algorithms in this domain is k-NN. Later, kth-NN was proposed, which is the enhanced version, by addressing the flaws in the original k-NN algorithm.

4.4.4 Clustering-Based Anomaly Detection Approach

In the *clustering* approach, if the data point fits into a cluster, it is considered a normal point; otherwise, it would be an anomaly. That is to say, the expected outcome is binary (anomalous or non-anomalous), and there is no quantitative scoring measure to determine how strong or weak the anomalies are being classified.

²e.g., Gaussian distribution

4.4.5 Isolation-Based Anomaly Detection Approach

The *isolation* approach works well for imbalanced data sets whereby the anomalies are limited compared to the normal points, making it easier for algorithms to isolate the anomalous points. Isolation forest is the earliest algorithm in this domain, and over the years, researchers proposed enhanced algorithms (e.g., SCiForest, HS-Tree) to overcome the shortcomings of the isolation forest method.

4.4.6 Ensemble-Based Anomaly Detection Approach [40]

The notion behind the *ensemble* technique is to combine multiple independent anomaly detectors, which would be beneficial to obtain a better prediction result (better performance). Different models like Logistic regression, Support Vector Machine (SVM) or random forest can be integrated to form an ensemble. The two widely used ensemble techniques are: (1) Bagging and (2) Boosting. These techniques can address classification and regression problems.

4.4.7 Subspace-Based Anomaly Detection Approach

In the *subspace* approach, the aim is to detect the anomalies where they are hidden in the subspace of feature data. For example, in high dimensional data, a subspace is selected and within that subspace, the anomalies will be detected.

Methods	Advantages	Disadvantages
Statistic model	+ It is a simple and intuitive method	- It is computationally expensive if the given dataset is multi-dimensional ³ . - The method is incapable of detecting anomalies, if the anomalies are dependant to more than one feature in the dataset - It is computationally expensive for non-parametric algorithms (like kernel based) to be performed
Density	+ It is an intuitive and widely used method + The overall performance of density method is higher than <i>statistical</i> and <i>distance</i> based approaches	- It is computationally expensive - It is not recommended for high dimensional and large scale dataset - It is sensitive to parameters (e.g., if the size of k (nearest neighbors) is large)
Distance	+ It is straightforward and easy to apply + It is a data-driven approach in which the distribution of data does not have any influence on this method (unbiased to the distribution of data)	- The time complexity for distance-based methods is generally high
Clustering	+ It is suitable for unsupervised learning ⁴ + It is suitable for complex data set	- The time complexity is significantly high - It is sensitive to the given parameters
Isolation	+ It has a low running time ⁵	- Tree-based algorithms (e.g., iForest) is incapable to detect local outliers in the data set
Ensemble	+ It is highly scalable + It is suitable to be used when the data set has global and local anomalies + It is highly scalable	- In comparison to isolation-based method, ensemble approach has a higher time complexity
Subspace	+ It performs better in comparison to a single anomaly detection model + This method is suitable to detect the hidden anomalies	- It has a high time complexity ⁶ - The method is susceptible to unrelated attributes in the data set

Table 4.1: Comparison of anomaly detection techniques [34]

³it is more suitable for univariate data⁴it is often a suitable approach when the feature labels of the data set are not given and by plotting the data using clustering approach, one could visualize how the distribution of the data and anomalous points are located⁵the time complexity is relatively low⁶the time complexity is comparable to nearest neighbor-based techniques like density or distance approaches

4.5 Anomaly Detection methods in the Fraud Detection domain

Anomaly detection methods in the fraud detection domain, can be categorized into three areas: (1) statistical, (2) machine learning, and (3) deep learning methods [3].

4.5.1 Detecting anomalies based on the statistical approach

Hashemi [3] has suggested three methods on how to handle the outliers in *statistical* approach: (1) *Modified Z-score*, (2) *Interquartile range*, and (3) *Histogram bins*.

The intuition behind *Z-score* method is by calculating the standard deviation of the data point to the mean of the rest of the data points. A significant value indicates the data point is anomalous in a data set. *Z-score* is based on the notion that the distribution of data points is normal [13]. However, since the outliers are not normally distributed in anomaly detection, *Z-score* is prone to outliers in the data set. Therefore, as [13] recommended, *modified Z-score* would be an optimum approach whereby it uses the median absolute deviation.

The observation points divide into four sections in the *Interquartile range* method. It will be considered an anomaly if the data point is not within the first to third quartile. This method can be adjusted to *intertentile* range, whereas the *nth* tentile will be used to label the data points.

Histogram bins method is a visualization approach whereby the histogram of a variable is formed, and the bins with higher heights indicate the anomalous points.

One of the shortcomings of the statistical approach in anomaly detection is the incompetency to handle the categorical data [3].

4.5.2 Detecting anomalies based on the machine learning approach

Contrary to the statistical approach, machine learning methods can handle categorical data. Hashemi [3] has suggested five machine learning algorithms that can address anomaly detection problem in the fraud detection domain: (1) *k-nearest neighbors (kNN)*, (2) *One-class SVM (OC-SVM)*, (3) *DBSCAN*, (4) *LOF (Local outlier factor)*, and (5) *Isolation Forest*.

KNN is a distance-based anomaly detection method (see section 4.4.3). The intuition behind the *kNN* algorithm is based on two types of observations: *similar observations* are within close proximate ratio to each data point; and *lonely observations* indicates outliers, which are farther from the rest of the data points.

The intuition behind the *OC-SVM* method is that by compiling a model during a training process, an algorithm identifies and classifies the anomaly points in the test data as *out-of-class* if the data point lies outside of the boundary from the normal class ⁷. Two approaches suggested by Schölkopf [35] and Tax and Duin [38]. Schölkopf et al. [35] proposed using a plane in *n*-dimensional feature space (also known as hyperplane). In the second approach, Tax and Duin [38] suggested using a hypersphere to identify anomalous points.

DBSCAN is a density-based approach (see section 4.4.2) where data points with low-density areas identify as anomalies with respect to the normal points that are located in the densely packed regions. It is a popular unsupervised learning approach. As for the tuning process, there are two hyperparameters involved in this method: (1) *epsilon* sets the distance boundary for the cluster that data points that can be included, and (2) *min_samples* sets the minimum number of data points in a cluster.

⁷the difference between a normal SVM and one-class SVM is that in OC-SVM only one class is being used

Similarly, *LOF* is a density-based approach, and it identifies local outliers based on the neighborhood's density. If a point is far from the highly dense cluster, it is considered an outlier. While *LOF* works well with identifying local outliers, this approach is not well-suitable for global objects [9].

One of the most suitable approaches to identify anomalies in a large-scale data set (Big Data) is *isolation Forest (iForest)* [44]. The time complexity of the method is linear, and it works well to address the high dimensional problems [26].

4.5.3 Detecting anomalies based on the deep learning approach

Autoencoder

Autoencoder is a deep learning method whereby, the output of the reconstructed data is given by the input data. As shown in Figure 4.1, this reconstruction is performed using two smaller networks. There are two main processes in the Autoencoder approach; first, the encoder learns a set of features based on the given input data, and then the decoder uses it to reconstruct the data based on the learned features.

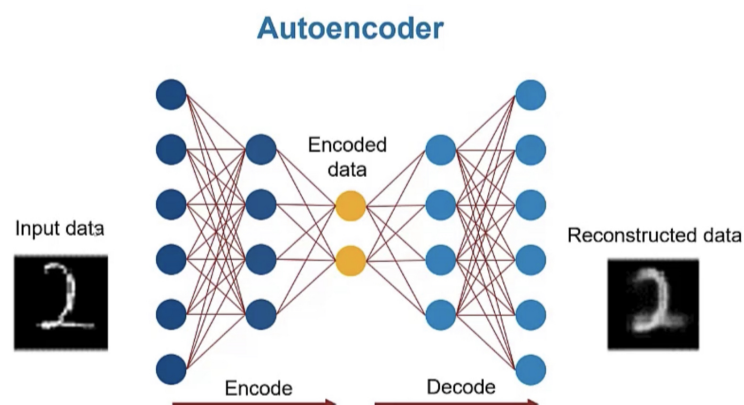


Figure 4.1: Autoencoder Network [23]

The Autoencoder model identifies anomalies by reconstructing the errors based on each data point. That record is considered an anomaly if these errors exceed a defined threshold based on the contamination value.

4.6 Conclusion

Anomaly detection is the course of action to identify any abnormal behavior of the system. It tends to address the problems related to machine learning or data mining domains. The methods to detect anomalies categorize into seven fields, each with weaknesses and benefits. That is to say, applying a suitable technique for anomaly detection problems is highly dependable on the specific use case, application domain, and the nature of the data such analysis is being conducted.

5 Anomaly Detection of Wireless Sensor Network data using Machine Learning Techniques

5.1 Introduction

This chapter describes different types of anomaly detection techniques based on the availability of labeled data in section 5.2. Section 5.3 introduces the use case of this research study. Next, section 5.4 discusses the research design and the anomaly detection pipeline specifically defined based on the use case study. Furthermore, section 5.5 covers the data analysis of processed data. In chapter 6, the obtained results from the models are discussed in two compartments: (1) Fraud Detection and (2) Time series Analysis, followed by the conclusion in chapter 7.

5.2 Labelled data in Anomaly Detection techniques

With the help of data labels, anomaly detection algorithms could determine if the transaction is to be considered normal or anomalous. However, choosing the right set of labels to be used for training the data set is a challenging task, as it requires domain expert knowledge to observe various behaviors to perceive which behavior is considered anomalous. In this context, if the anomalous behavior is dynamic and the new behavior has not been included in the training phase of the model, it is most likely that the anomalous transaction will not be identified as an outlier. For this reason, three different techniques can be used solitarily for anomaly detection based on the availability of the labels in a given data set.

Figure 5.1, shows which Anomaly Detection techniques would be the most appropriate based on the availability of the labeled data.

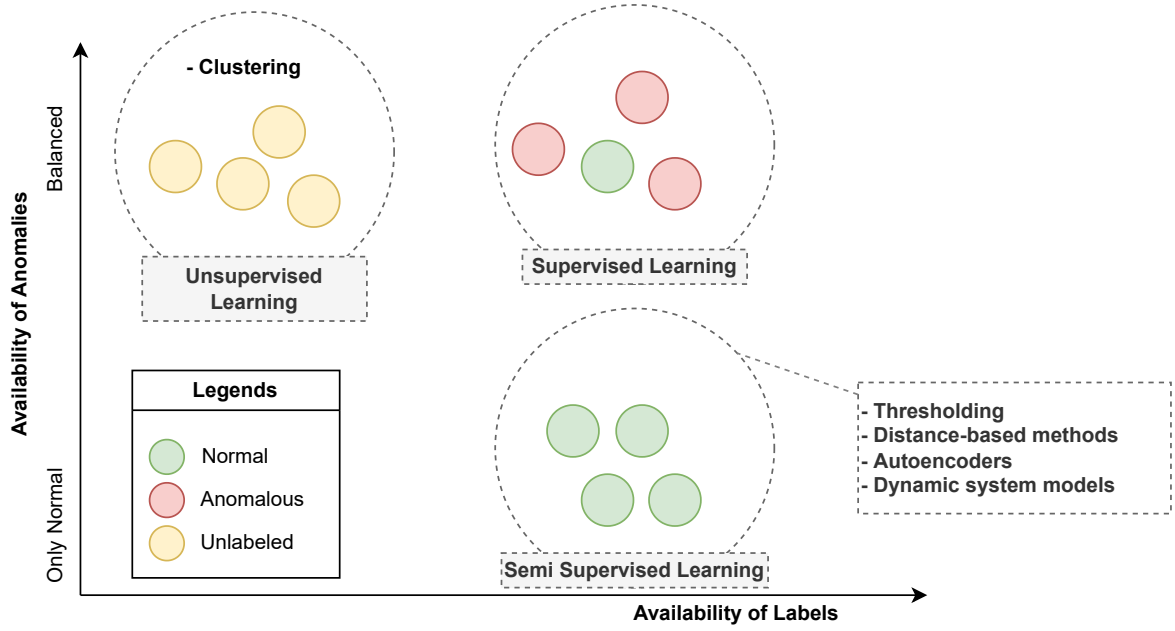


Figure 5.1: Choosing the right Anomaly Detection technique based on a given amount of labeled data available [23]

5.2.1 Supervised Learning for Anomaly Detection

Supervised learning AD can be best described as when the label data are well defined in the data set; an algorithm can use the data to train a normal model and then make predictions, including identifying anomalies. Generally speaking, since the proportion of anomalous data in the training data set is considerably lower than the normal data, learning the abnormal behaviors for supervised algorithms is a limitation for imbalance training data set.

5.2.2 Semi supervised Anomaly Detection

In contrast to supervised learning (section 5.2.1), semi-supervised AD tends to train a model using only normal class data set. Identifying all possible anomalies in the training data set might lead to overlooking or prone to failure of anticipation. However, there are still possible solutions for semi-supervised learning by detecting the anomalies in test data, using a normal class model [10]. Prominent semi-supervised algorithms are ocSVM (One-Class Support Vector Machine) [36], LOF (Local Outlier Factor) [8], kNN (k-Nearest Neighbor) [19], and Autoencoder [28]. Autoencoder has been selected as the initial part of this research study to identify the anomalies. Then, the predicted results were incorporated to form a classification modeling using the Logistic Regression approach (section 6.2.2). The detailed workflow pipeline will be discussed in section 5.4.

5.2.3 Unsupervised Anomaly Detection

Lastly, in unsupervised learning of anomaly detection, the notion would be that the data set that will be used for the learning process does not contain a high risk of detecting anomalies in the data. This led to a high number of false positive alarms to occur. On the contrary,

unsupervised learning help to cover a more extensive range of possibilities to identify anomalies. More precisely, as discussed in the supervised learning approach, the model's capacity to detect anomalies is limited to the dataset which is being trained.

5.3 Use case of this study

The use case of this study falls under anomaly detection and fraud detection mechanism whereas, with the help of real sensor data, the model could predict the authenticity of the sensor records communicating with the base station (the base station refers to cloud computing resource that can perform fact checking on the records before the data can be stored in the database).

5.3.1 Feature data and types of sensors

The dataset for this research study is based on the Proteus level sensors founded by Inno-Tec GmbH [32]. The filling level of various types of tanks, such as heating oil, water, cistern, gasoline, or rain barrel, can be measured with the help of proteus level sensors. After the sensor has been securely authenticated to the cloud (base station), the measured filling levels will be sent to the base station. Table 5.1 highlights various datasets that were collected for this study. These datasets can be categorized based on their applications, sensor types, and filling behavior patterns. For instance, the filling behaviors of heating oil and used oil tanks act as follows: For heating oil, the tank becomes full and decreases based on consumption. When the tank's temperature increases, it causes the oil to be heated and expands. Therefore, the oil's relative density decreases. In contrast, once the tank becomes full of used oil, the supplier's support personnel should be on-site to empty the tank.

Sensor Type	Tank Type	Filling Behaviour
Ecocloud	AdBlue	Tank level falls until it is getting filled up
Ecocloud	Gasoline	Tank level falls until it is getting filled up
KFA5	Heating Oil	Tank level falls slowly until it is getting filled up
KFA5	Used Oil	Tank level rises until it becomes emptied
Ecofrog	Water	There is no standardized pattern behaviour, as the tank can be filled in by the rain (depending on the weather condition) or consumed for gardening in irregular intervals)

Table 5.1: Datasets of Proteus level sensors

The dataset selected to perform this report's analysis is based on KFA5 sensor data. Table 5.2 describes the fields used for data modeling process:

Field name	Type	Field description
PrimaryKey	STRING	Unique value generated for every transaction
TankID	STRING	Distinguish value for each tank
DeviceID	INTEGER	Distinguish value for each sensor device
ProductID	INTEGER	This value identifies the sensor type. For instance, for KFA5 records, this field would be identical across different tank types (used oil, or heating oil)
DateTime	TIMESTAMP	Timestamp of the measured filling level for each transaction
Temperature_C	INTEGER	Temperature of the sensor in Centigrade
Temperature_F	INTEGER	Temperature of the sensor in Fahrenheit
ULL1	FLOAT	ULL1 refers to the amount the tank is falling short of being refill to the fullest amount
AMPL1	FLOAT	AMPL1 refers to the amplitude of the filled level corresponding to ULL1 value
ULL2	FLOAT	ULL2 refers to the amount the tank is falling short of being refill to the fullest amount
AMPL2	FLOAT	AMPL2 refers to the amplitude of the filled level corresponding to ULL2 value
ULL3	FLOAT	ULL3 refers to the amount the tank is falling short of being refill to the fullest amount
AMPL3	FLOAT	AMPL3 refers to the amplitude of the filled level corresponding to ULL3 value
SIGMIN	INTEGER	Minimum sigma value
SIGMAX	INTEGER	Maximum sigma value
SIGSTD	INTEGER	Sigma (standard deviation) value
CorrMess	INTEGER	CorrMess values vary between 1,2 and 3. If any value from these three numbers is assigned, that means the corresponding ULL and AMPL have been chosen for the most accurate filling level

Table 5.2: KFA5 dataset properties

It is worth mentioning that, when this research was conducted, only specific device IDs were known to be in an active state¹. The list of the active sensors is shown in Table 5.3:

¹active state refers to the sensor devices that are precisely mounted into the tank

Active SensorID	DeviceID	Tank Type
KFA5-07	867198051290751	Used Oil
KFA5-09	867198051284887	
KFA5-11	867198051290405	Heating Oil
KFA5-16	867198051290561	
KFA5-17	867198051290496	Used Oil
KFA5-18	867198051291247	
KFA5-20	867198051284879	

Table 5.3: KFA5 Active Sensors

Therefore, the accuracy of the remaining transactions for non-active sensors could be naturally susceptible to error (error-prone). For this reason, a new column (ActiveSensorClass) was created and added to the data set to classify, compare and distinguish the valid and unconvincing transactions.

5.4 Detailed view of the research design

The focus of this research study can be broken down into two main areas: (1) Anomaly Detection in Fraud Detection Analysis (section 5.4.1); and (2) Anomaly Detection in Time Series Analysis (section 5.4.2). The platforms used for this study rely on the Google Cloud service (BigQuery ML). The proposed solution for the fraud detection mechanism is a semi-automated approach, whereas, in Figure 5.2, the high-level workflow for the Anomaly Detection pipeline is illustrated.

As described in section 5.2, the methodology used for the fraud detection mechanism commences with the semi-supervised anomaly detection method, whereby the Autoencoder method is used to determine the first round of anomalies. Next, in the initialization phase, the Autoencoder predicted results indicated each transaction's predicted feature (as anomaly or non-anomaly). Next, that response feature label has been used for binary classification learning. Once the pre-training (initialization) phase is performed to address the imbalanced data classification problem, the data set is being re-trained with the Logistic Regression Classification method and the output results generated by Logistic Regression models used for evaluating the performance of the models for detecting anomalies. Time-series analysis for detecting anomalies was conducted in the second part of this research study.

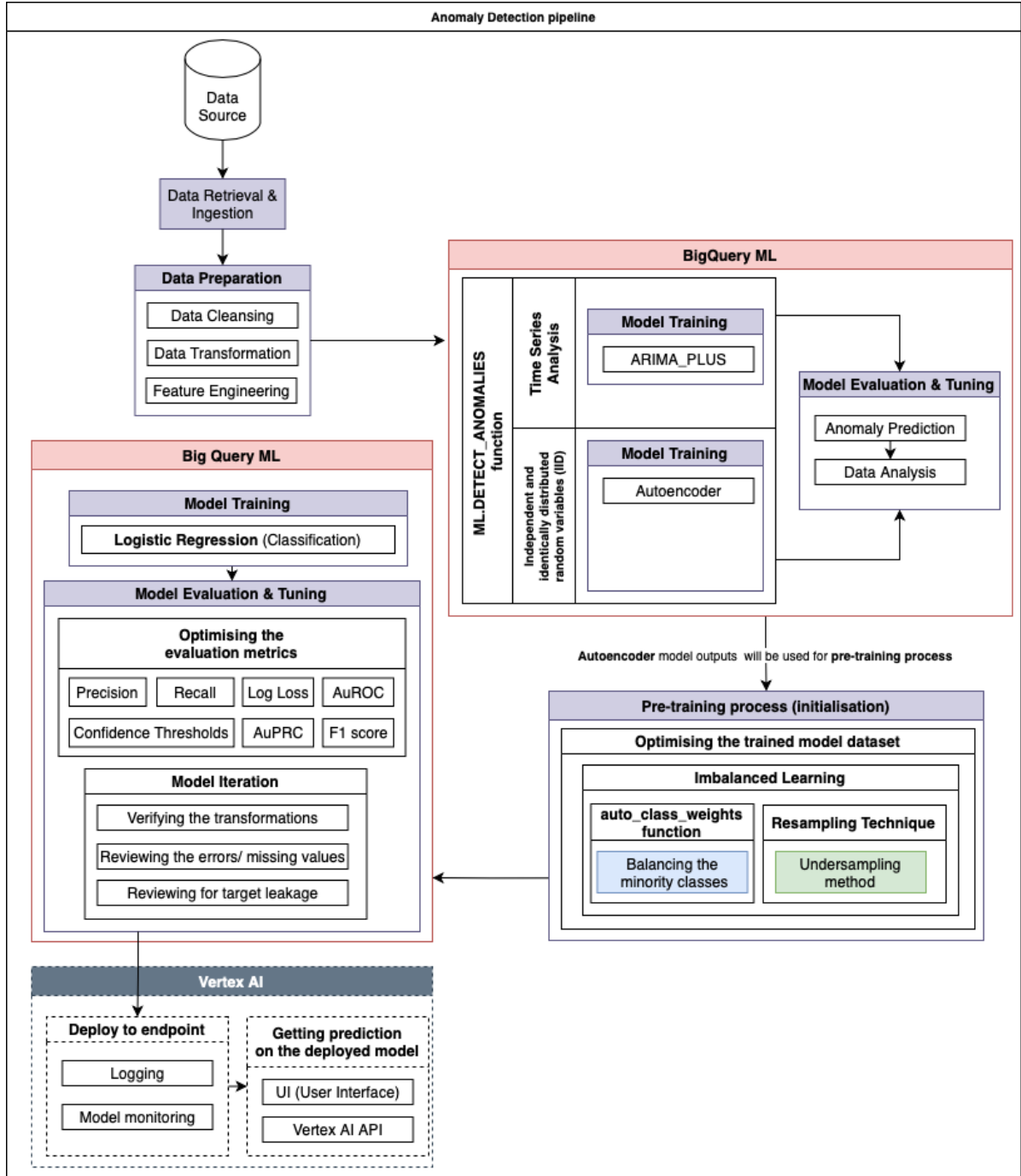


Figure 5.2: Anomaly Detection Pipeline

5.4.1 Anomaly Detection in Fraud Detection Analysis

As described in section 5.4, the proposed fraud detection mechanism consists of three main phases: (1) Training a model using Autoencoder; (2) Pre-training (initialization) phase; and (3) Training a new model using classification learning approach (Logistic Regression).

Training a model using Autoencoder in BigQuery ML

As described in section 5.2.1, the intuition to choose the Autoencoder method for the initial phase of the fraud detection mechanism was based on three cogitations: firstly, since the initial dataset did not contain a target feature label to identify if the records are having normal or abnormal behavior, semi-supervised learning method in particular Autoencoder was used to generate the response label. The second reason was to avoid the limitation of the imbalance constraint of training data which was discussed in Supervised Anomaly Detection learning (section 5.2.1), and to ensure all records are being fully utilized for the later stage of the classification phase (including the minority classes). The third reason was to ensure there were no human biases involved in the decision-making process of classifying the target column of the transactions.

It is worth mentioning that Autoencoder is based on the IID (Independent and Identically Distributed random variables) probability theory whereby, *“a collection of random variables is independent and identically distributed if each random variable has the same probability distribution as the others and all are mutually independent”* [16]. The intuition behind *identically distributed* is based on the notion that all the collected samples are from the same probability distribution. In other words, there is no fluctuation concerning the distribution. Moreover, *independent* characteristic means the events (transactions) are not connected explicitly. That is to say, having an insight into the value of one variable does not release any information containing the value of other variables.

Understanding IID probability theory is essential because the compiled dataset to perform semi-supervised learning (using Autoencoder) is the collection of KFA5 Heating and Used Oils transactions with the notion in mind that each transaction is being evaluated independently. More specifically, combining the filling behavior of heating and used oils into a single dataset to train an Autoencoder model does not disrupt the model prediction since each transaction is being treated separately.

Pre-training (initialisation)

In this phase, the predicted results from the Autoencoder model are studied to evaluate the proportion of class distribution of the records to address the classification problem, which is discussed in the next chapter with a more concrete view. Two solutions (weighting and resampling) were used to address the imbalanced learning problem.

Training a model using classification approach (Logistic Regression)

The prerequisite for the binary classification is to have a pre-determined target feature column which makes it possible for the model to provide a probability score for prediction. Based on the use case, the target column represents the anomaly class generated from the Autoencoder model. In addition, the optimization objective has been set to AUC PRC to achieve the maximum precision-recall for the less common class.

5.4.2 Anomaly Detection in Time Series Analysis

Another aspect of anomaly detection is analyzing time-series records. ARIMA_PLUS model addresses the scenario, whereas the model can detect the number of transactions received from the sensor devices within a certain threshold. This mechanism is beneficial to prevent the base

station from a denial-of-service attack (DoS attack) or distributed denial-of-service attack (DDoS attack). In addition, it acts as a safe gateway for detecting anomalies.

5.5 Data Analysis

The data analysis of this research covers three main compartments: (1) analysis of the processed data for model creations covers in section 5.5.1; next, in chapter 6 (Data Modeling and Results), (2) analysis of anomaly detection based on the predicted results for fraud detection mechanism covers in section 6.2.1; and (3) time series analysis addresses in section 6.2.3.

5.5.1 Data analysis of processed data

The data set used for the training phase contains 53,295 records and 21 feature columns in conjunction with the listed distinct values (Table 5.4). The objective of this section is to analyze different feature values like ullages and temperature data with the intention of identifying anomalies. The time frame of collected data is between January 12th, 2021, and April 23rd, 2022. Furthermore, to minimize the noise, the following graphs only present active KFA5 sensor records as listed in Table 5.3.

Field name	Distinct values
PrimaryKey	53295
TankID	58
DeviceID	59
ProductID	1
DateTime	31954
Temperature_C	28
Temperature_F	28
ULL1	5921
AMPL1	41239
ULL2	6108
AMPL2	37255
ULL3	6283
AMPL3	31909
SIGMIN	1348
SIGMAX	1505
SIGSTD	521
CorrMess	3
ActiveSensorClass	2

Table 5.4: Processed data schema for model creation

Data Analysis of Ullage measured space values in KFA5 Sensors

As described in table 5.2, and shown in figure 5.3, ullage volume determines from the point where the sensor is located on the tank lid to the surface of the oil level. Once the tank is refilled to the fullest amount, ullage space becomes borderline to zero. In theory, an ultrasonic sensor initiates the process by transmitting sound, and once it hits the surface (e.g., oil level), the

receiving sound can determine the ullage space. In the use case of this study, the proteus level sensor contains three distinguish ullage measurements (ULL1, ULL2, ULL3). That means the ullage volumes are measured from three distinguished angles (or locations) in the tank.

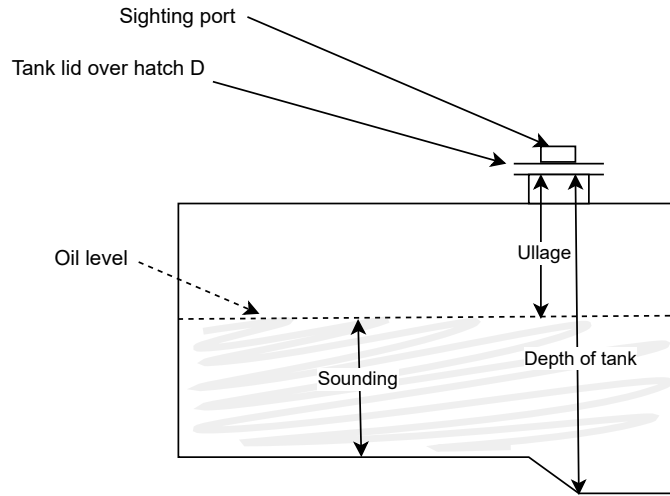


Figure 5.3: Ultrasonic Level Diagram

Figure 5.4, and figure 5.5 illustrate the aggregated values of ullage 1 and ullage 2 for KFA5-17 and KFA5-16 sensors.

Figure 5.4 highlights three *point anomalies* and two *collective anomalies*. The pattern shows that by removing the *point anomalies*, the maximum values for the aggregated ullages are nearly constant (approximately at the total value of 1.5). Once the aggregated ullages reach the maximum value, they decrease until the tank is refilled. The *collective anomalies* shows at which intervals the aggregated ullages were not decreased according to the pattern.

Furthermore, figure 5.5 highlights four *point anomalies* and three *collective anomalies*. Similar to figure 5.4, the graph shows that by eliminating the detected anomalies, the maximum values of the aggregated ullages become close to 1.5.

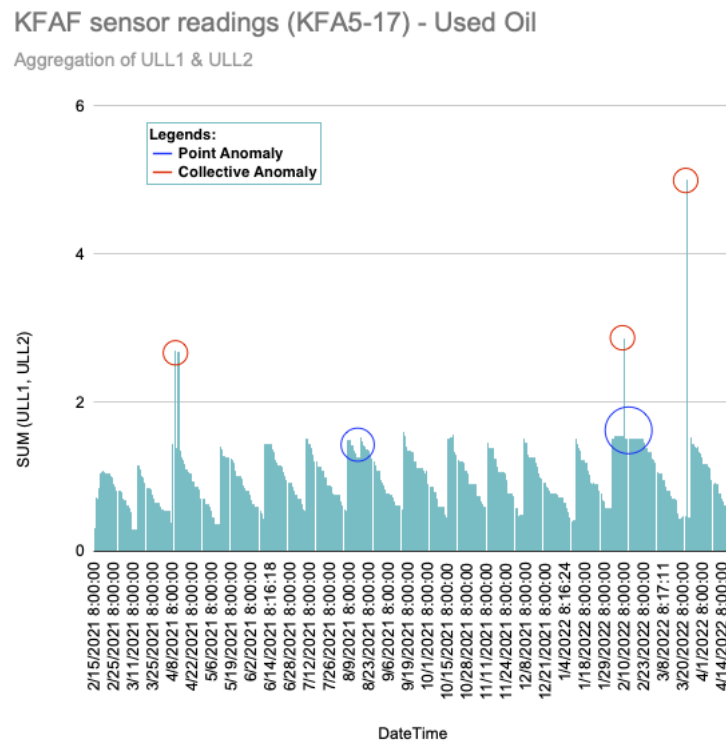


Figure 5.4: Aggregated Ullage Measured Values - KFA5-17 Sensor Device

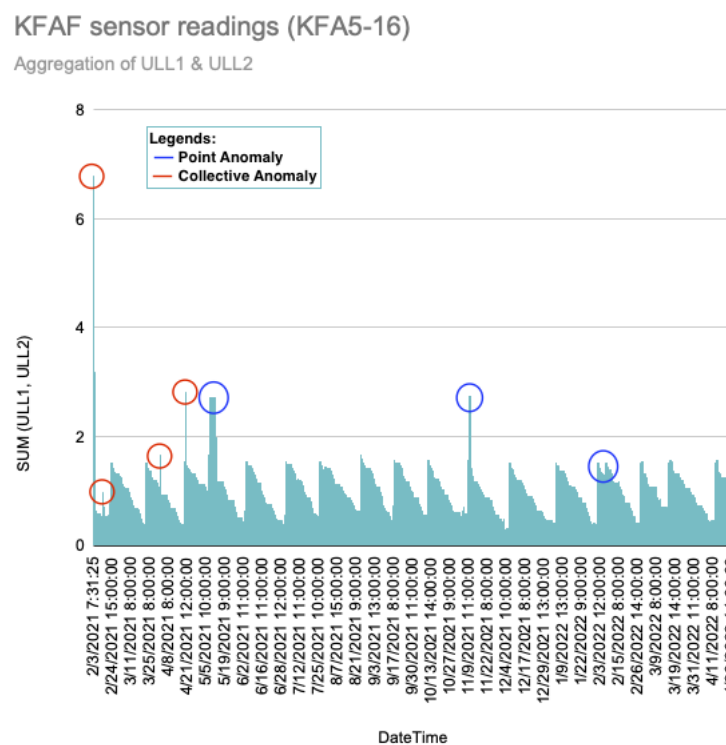


Figure 5.5: Aggregated Ullage Measured Values - KFA5-16 Sensor Device

Data Analysis of Temperature measured values in KFA5 Sensors

Figure 5.6 shows the *point anomaly* on the left side of the graph when the temperature drops to 12 Celcius. This point deviates from the rest of the data points for this sensor. In addition, since the temperature level does not change much over the period of one year (from 04/01/2021 until 04/01/2022), this indicates that the sensor is located in an indoor environment. On the contrary, figure 5.7 indicates the sensor is located in an outdoor environment due to the fluctuation.

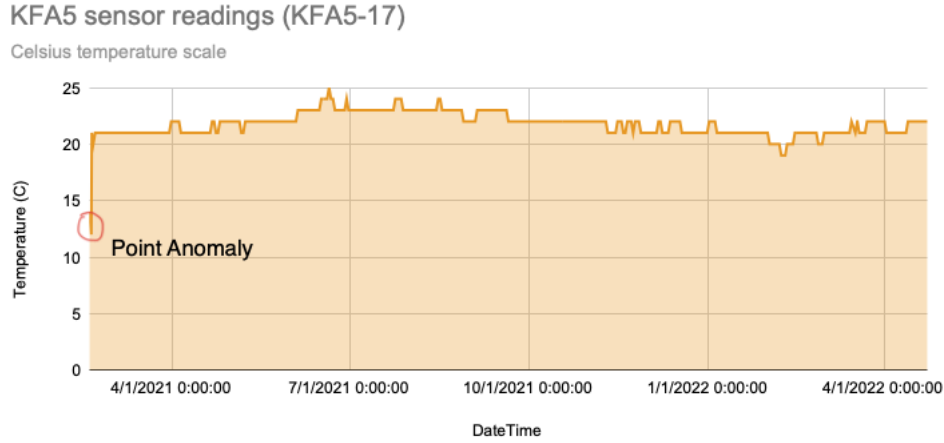


Figure 5.6: Temperature Measured Values - KFA5-17 Sensor Device

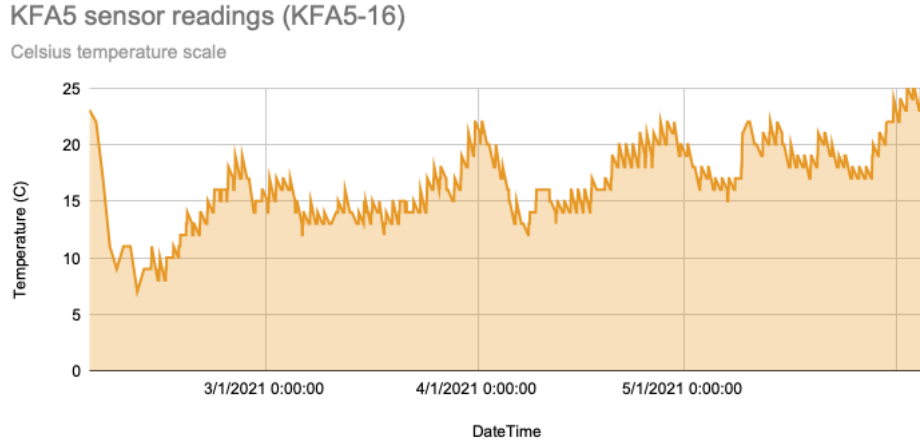


Figure 5.7: Temperature Measured Values - KFA5-16 Sensor Device

The third graph (figure 5.8) highlights the *contextual anomalies* on the left corner. There could be various reasons for this abnormality, but one assumption could be if the sensor was directly exposed to the outside temperature. For example, one prominent scenario would be if the door of a titan garage (building) was opened during that timeframe.

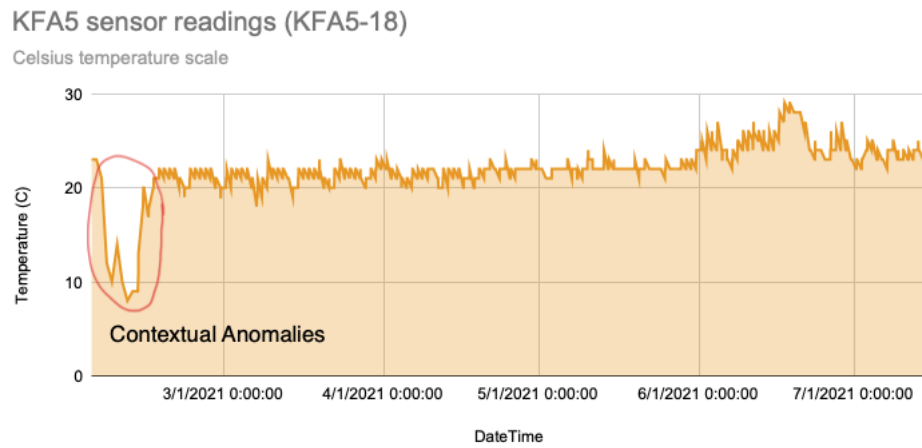


Figure 5.8: Temperature Measured Values - KFA5-18 Sensor Device

6 Data Modeling and Results

6.1 Introduction

This chapter discusses the methods used for data modeling. Section 6.2 articulates the results from the created models. Seven models are compiled for this study; three belong to the fraud detection analysis, and four belong to the time series analysis. Section 6.2.2 discusses different techniques to handle imbalanced data. Next, section 6.2.2 outlines the evaluation and performance of classification models for fraud detection. Lastly, section 6.2.3 studies four scenarios for time series analysis.

The data model pipeline consists of six phases:

Phases	Description
01_raw	The raw source file (without modifications) stores in this directory
02_intermmmediate	The modified raw dataset (with correct format) before processing and training a model stores in this directory (this data is the cleaned version of raw dataset)
03_processed	The processed data for model creation stores in thi directory
04_models	Generated models store in this directory
05_model_output	The predicted generated results from the models store in this directory
06_reporting	The final data for reporting and analysis store in this directory

Table 6.1: Data modeling pipeline

6.2 Data Models

As described in section 5.4 (Detailed view of the research design), the analysis for anomaly detection in this report covers two main compartments: (1) Anomaly Detection for Fraud Detection Analysis; and (2) Anomaly Detection for Time Series Analysis. This section discusses the results generated from anomaly detection models. The instruments used for building these anomaly detection models are from Google Cloud BigQuery platform and as for the graphical data representations, Connected Sheets¹, Data Studio, and Matlab have been used. The corresponding source codes can be found in the Appendix section of this report.

¹Connected Sheets is the integrated data analysis and visualization tool that is linked with BigQuery data, provided by Google Cloud [41]

6.2.1 Fraud Detection Data Models

As discussed in section 5.4.1, the fraud detection pipeline consists of two machine learning approaches: (1) Autoencoder (semi-supervised) method has been used to obtain the first round of predicted anomalies, and (2) Logistic Regression (supervised) method has been incorporated to form a binary classification for detecting anomalies of transactions. The intention to set up the fraud detection pipeline in this manner was to use semi-supervised learning (Autoencoder) to obtain the Target values² for the given data set (KFA5 sensor data) and then to train the generated dataset from the Autoencoder to perform supervised classification learning. Therefore, in this sub-section, at first, the Autoencoder model and its results are discussed (section 6.2.1); next, the results from Logistic Regression models are uttered (section 6.2.2); and last but not least, the results from the aforementioned models are compared and evaluated.

Autoencoder Model

Autoencoder is an artificial neural network (ANN). It is a useful method for unsupervised and semi-supervised learning to detect anomalies, specifically for imbalanced classes in the data set. The Autoencoder handles the random variables as independent and identically distributed. Therefore, the frequencies of the classes in the data set do not influence the model prediction. However, the imbalanced data is still a challenge for solving a classification problem which is discussed in section 6.2.2.

There are some advantages to using the deep learning approach in fraud detection use cases compared to the conventional approaches. First, incremental learning helps the model robust to the retraining process. In contrast, it does not require retraining the entire model for model optimization, which leads to the enhancement of computing power and overall model size.

As discussed in section 4.5.3, three main elements need to be considered during the model creation phase: (i) input/output layers, (ii) hidden layers, and (iii) neurons. This section discusses what values have been selected for these elements in addition to the available options in BigQuery ML to form an accurate model while avoiding overfitting or underfitting problems.

Autoencoder is one of the popular methods when it comes to dimensionality reduction. The dimension refers to the feature available in the data set, and when the data set contains multiple features, dimensionality reduction methods help to reduce these features, which could help for better performance³ of the model. However, by adding multiple hidden layers, the computational time will increase significantly, and overfitting might occur. Overfitting occurs when the model might be fitted with the training data set but not perform well for the test data. In Autoencoder, the model calculates the reconstruction errors, and if these errors are more significant than the defined threshold, it detects those data points as abnormal (anomalies) behaviors [17].

Table 6.2 outlines the defined parameters used for Autoencoder model. Since Autoencoder is considered a non-time-series approach, it was decided to drop the DateTime feature data for the model creation. The contamination value lays the threshold, and if the obtained reconstruction error surpasses this value, the algorithm identifies it as an anomaly. The contamination score has been set to 0.02 [31]. Following features have been selected to be included for the Autoencoder model: DeviceID, ULL1, AMPL1, ULL2, AMPL2, SIGMIN, SIGMAX, SIGSTD, ActiveSensorClass, TankID, Temp_C, Temp_F, ULL3, AMPL3, FM, Batt, Crssi, CorrMess.

²target value is referring to the predicted result as a Boolean representation whereas, if the transaction has been predicted as normal (False) or anomalous (TRUE)

³the performance in this regard, is referring to computational part of model creation

Based on the observation, in feature engineering, the above selected features for Autoencoder model provides lower⁴ mean_absolute_error, mean_squared_error and mean_squared_log_error scores. These parameters are discussed in table 6.3.

Options	Selected Arguments	Description
model_type	Autoencoder	
activation_fn	relu	RELU refers to Rectified linear
batch_size	8	
dropout	0.2	
hidden_units	[32,16,4,16,32]	
learn_rate	0.001	
l1_reg_activation	0.0001	
max_iteration	10	
optimizer	adam	

Table 6.2: Autoencoder parameters

BigQuery ML calculates the reconstruction losses between the input and output data of an Autoencoder model [39]. The output of the reconstruction loss function contains three parameters: (1) mean_absolute_error, (2) mean_squared_error, (3) mean_squared_log_error. Table 6.3 outlines these parameters alongside with the predicted feature class (is_anomaly column).

Feature columns	Description
is_anomaly	Boolean representation (TRUE/FALSE)
mean_absolute_error (MAE)	MAE is a form of representation, by comparison, predicted data against the observed one. It is determined by dividing the sum of absolute errors ⁵ over a given sample size
mean_squared_error (MSE)	MSE is determined by squaring the MAE score, which led to a positive (non-zero) score. MSE is a risk function indicator whereby, if there is a considerable difference (loss) between the predicted and observed data, the result of MSE will eventually increase exponentially
mean_squared_log_error (MSLE)	MSLE represents a ratio of true classes and predicted values using a logarithm function. MSLE treats small and big deviations in a similar manner. For instance, if there are two MSEs (MSE1 as 100 and MSE2 as 100,000,000), the calculated MSLE1 would be 0.02861, and MSLE2 to be 0.03100. That shows the delta of MSLEs would be small even though the delta of MSE losses is large [30]

Table 6.3: BigQuery ML reconstruction losses output function (Autoencoder)

It is worth mentioning that if the result from summing the absolute errors led to zero (or close to zero), that does not indicate the excellent accuracy of the model prediction, as the errors could be a mix of positive and negative values. Therefore, throughout this report, the performance and evaluation of the models highly rely on the MSEs scores.

⁴the intention is to have a lower reconstruction errors in which it indicates that the gap in between true classes and predicted labels are low

⁵errors are referring to the differences between the predicted and observed one

Figure 6.1 illustrates the above construction loss parameters with respect to all transactions from the test data. The red labels outline the transactions the Autoencoder model predicted as fraudulent, and blues are classified as normal transactions. Each transaction has a unique identifier stored in the PrimaryKey feature column and presented as Y-axis.

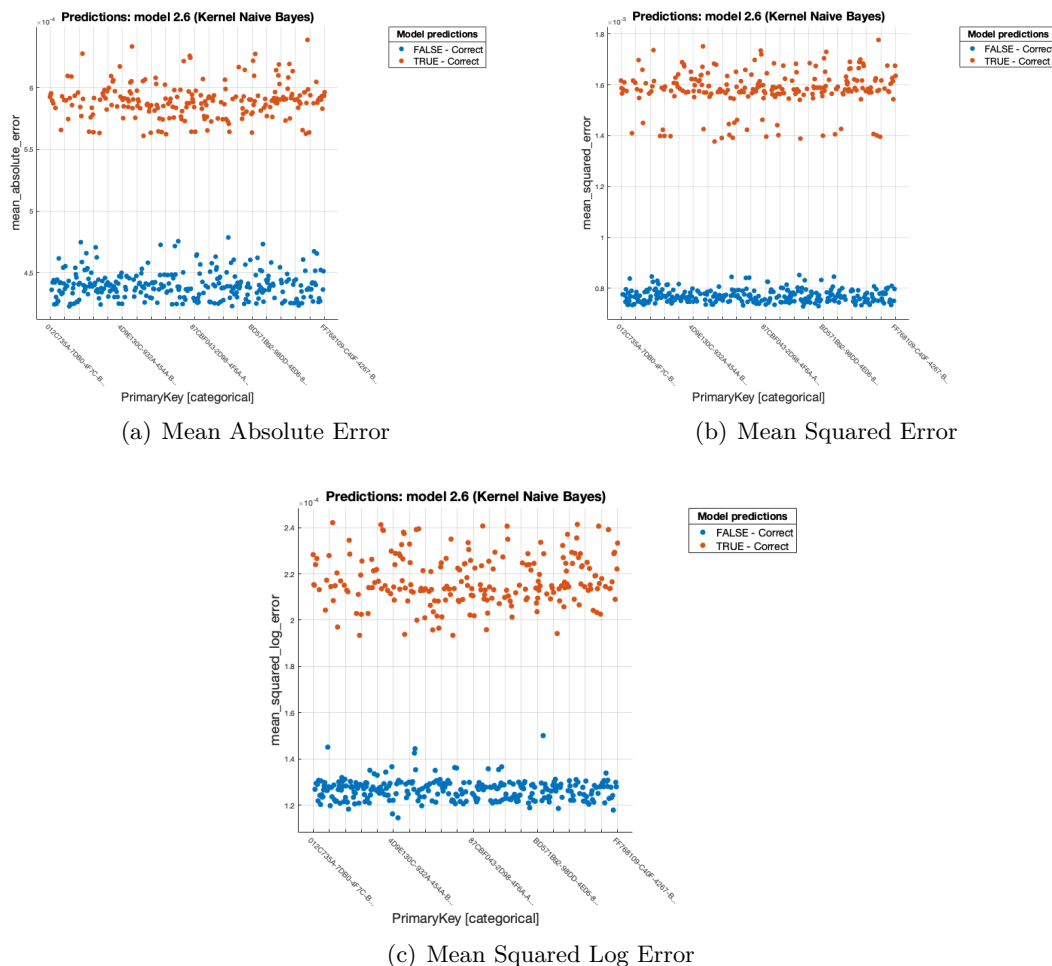


Figure 6.1: Reconstruction Loss - Autoencoder model

Autoencoder predicted results

Figure 6.2 illustrates four distinguish graphs. Figure (a) shows the cumulative trend of three reconstruction loss parameters. The total accumulated values for MAE, MSE, and MSLE are 17.1, 23.19, and 4.55, respectively. Figures (b), (c), and (d) demonstrate the error rates in a reverse cumulative distribution.

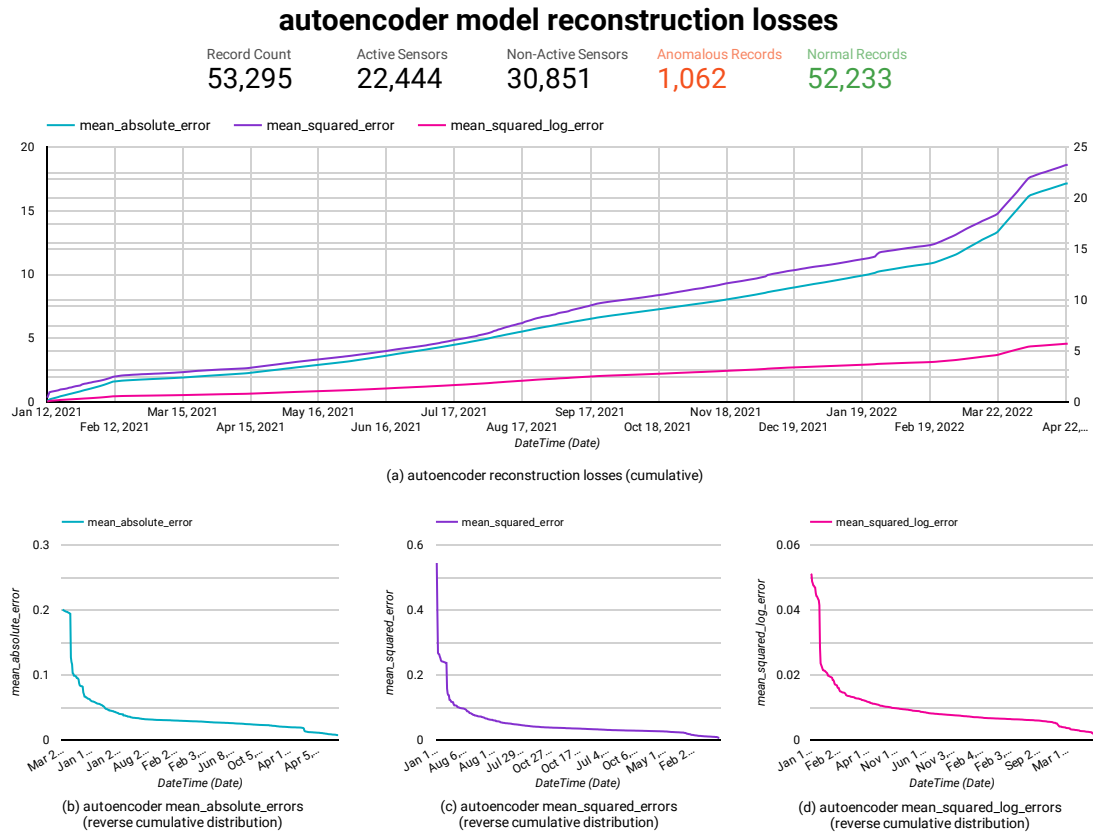


Figure 6.2: Autoencoder reconstruction losses trends

Table 6.4 provides the statistics of Autoencoder prediction results. By looking at the proportion of classes in the table, it can be observed that the initial data set that was used for Autoencoder modeling and prediction is imbalanced. This observation lays an important ground before proceeding with classification learning (the second phase of the fraud detection pipeline). Because of an imbalanced data set, classification algorithms tend to be biased with the majority class of the data during the training phase. In order to address the imbalanced learning problem, the next section details the techniques that have been used in this research study.

Sensor Type		Predicted label	
		Anomaly	Non-anomaly
Active	22,444	280	22,164
Non-active	30,851	782	30,069
Total	53,295	1,062	52,233

Table 6.4: Autoencoder prediction results

6.2.2 Handling the Imbalanced dataset [25]

An imbalanced problem occurs when the representation of classes in the dataset is not equally distributed. For example, one class has fewer or more records/transactions compared to other classes. This imbalance of data for a majority and minority classes raises an oversight for most algorithms during model creation. For example, the system could predict well for majority

classes but not accurately for minority classes. Classification learning is a well-known method that is afflicted by imbalanced datasets.

Although the imbalanced problem is a common problem in some applications whereby the number of normal records is likely to be a majority class of the dataset, imbalanced learning can help to address this problem. Furthermore, to the nature of the design, since frauds and anomalies are considered abnormal behaviors from the rest of the normal patterns, imbalanced learning methods have been used in this part of the study in order to be able to implement a fraud detection mechanism using classification learning approaches.

The techniques used for imbalanced learning can be implemented in two aspects: (1) using *resampling* technique, whereas the number of records will be manipulated according to the use case; and (2) *cost-sensitive* approach, certain logic will be set concerning how costly the wrong prediction class could impact on the system. For instance, the cost of predicting a false-positive class for diagnosing a cancer patient could be higher than the false detection of the weather forecast (false detection of weather forecast might not have a severe impact on the outcome compared to cancer patients). Depending on the business goal, the cost-sensitive approach could have a different perspective on model performance.

In imbalanced learning, *Resampling* and *cost-sensitive* approaches operate at different stages of machine learning workflow; *Resampling* methods operate at the data level, whereas during the preprocessing phase, the number of transactions will be adjusted and balanced. On the other hand, in *cost-sensitive*, the changes apply after the model creation and during the model optimization.

Resampling techniques

As described in the previous section, resampling operates at the data level, where the imbalanced dataset will be modified before the training phase (preprocessing stage). One advantage of the resampling technique is that the process does not impact which types of classifiers to use for the training phase. In other words, the resampling technique is independent of the desired classification method. The resampling technique can be categorized into three main approaches: (1) Under-Sampling; (2) Over-Sampling; and (3) the hybrid version that tends to combine the first two approaches to form a new dataset [2][25].

Under-Sampling In this approach, once the number of classes has been identified, based on the amount of minority class records, the majority class records will be obtained and combined with the minority class. For instance, among 200 records of normal transactions and 50 as fraudulent, the under-sampling method only picks 50 records from each class and combines them to form a new dataset. There are some advantages and disadvantages to this approach. One benefit would be that the dataset shrinks to a smaller size, which could improve the computational side of model creation or prediction and reduce the model's size. More precisely, in embedded systems, the devices have a computing power constraint and limited battery capacity to perform the processing phase. On the contrary, if essential records in the majority class are removed from the dataset during the under-sampling phase, that could impact the model prediction.

Over-Sampling On the other hand, over-sampling tends to reproduce the records from the minority class to balance it with the majority class. Several approaches have been proposed to achieve that, such as duplicating minority records. However, this approach's disadvantage is that the dataset's size will be considerably large if the proportion of the minority class is far less than

the majority class. As highlighted in table 6.4, the number of minority class (anomaly) is 1.99% followed by 98.01% for the majority class. There are a total of 53,295 records; by balancing the classes in this regard, the new dataset will become around 104,271, considering the minority records to multiply by 49. Data augmentation can be used as an alternative approach to the over-sampling method. One prominent method is SMOTE (Synthetic Minority Oversampling Technique), where the algorithm creates synthetic data points based on the initial data points. These synthetic data points differ slightly from the duplicated minority class data points. [37]

Using under-sampling to create a dataset As highlighted in table 6.4, KFA5 sensor data used for Autoencoder to identify the anomalies had imbalanced data problem with the ratio of 2% for anomaly class which is True and 98% that is False. Hence, for the under-sampling method, 1062 transactions from both classes (anomaly and non-anomaly) were selected to be used for the new balanced data set (`undersampling_dataset`). Among the 2124 transactions from the undersampling dataset, 1072 records belong to non-active sensors and 1052 from active sensors.

Logistic Regression Models

The data obtained from two logistic regression models will be discussed in this section. Logistic regression is supervised learning where the model can predict if the value falls within the given response data (binary or multi-class regression). The response data label (`is_anomaly`) is based on the predicted results from the Autoencoder model. Since logistic regression is considered a classification problem, an imbalanced learning technique has been contemplated to address this issue. With the help of binary logistic regression, the aim is to determine how well the models predict if the given or future transaction is fraudulent. BigQuery ML has been used throughout the course of this assessment.

There are two models created for the logistic regression approach: (1) `model_01` built with an imbalanced dataset with `'auto_class_weights'`⁶ enabled to prevent overfitting the model, and (2) `model_02` built with the under-sample dataset. The intention was to evaluate how well each model performs and which could produce more accurate predicted results, which can then be deployed in production.

Feature Importance

Figure 6.3 illustrates the top 10 features that had the most attribution during the training phase for both models. Figure (a) shows that TankID had the most influence on the `model_01` with a considerable gap in comparison to the rest of the features; while, in `model_02` (figure (b)), that proportion has been distributed more uniformly.

⁶ `'auto_class_weights'` is the function in BigQuery ML for Logistic Regression models to balance the class labels in the training data. This setting is not enabled by default, which treats the training data as unweighted. However, for `model_01`, since an imbalanced dataset was used, this function was configured for the model creation

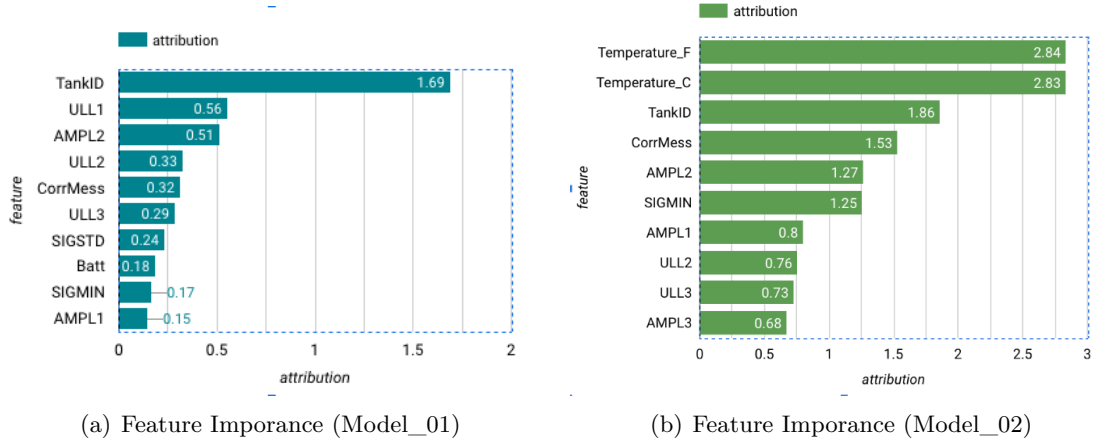


Figure 6.3: Feature Importance - Logistic Regression models (model_01, model_02)

Evaluation of the Logistic Regression models

Before the results for logistic regression models are discussed, some terminologies regarding model performance and evaluation metrics need to be articulated. A *precision* score helps to evaluate how well the algorithm has been used to predict the anomalies. This score refers to the real anomalies, and the goal is to have a higher score rate. A *recall* score helps to evaluate how well the algorithm identifies all anomalies. In a nutshell, the *precision* score covers the relevant anomalies, and the *recall* score covers the total relevant anomalies. Finally, a *precision-recall* (*PR*) curve illustrates a trade between precision and recall scores within the given threshold.

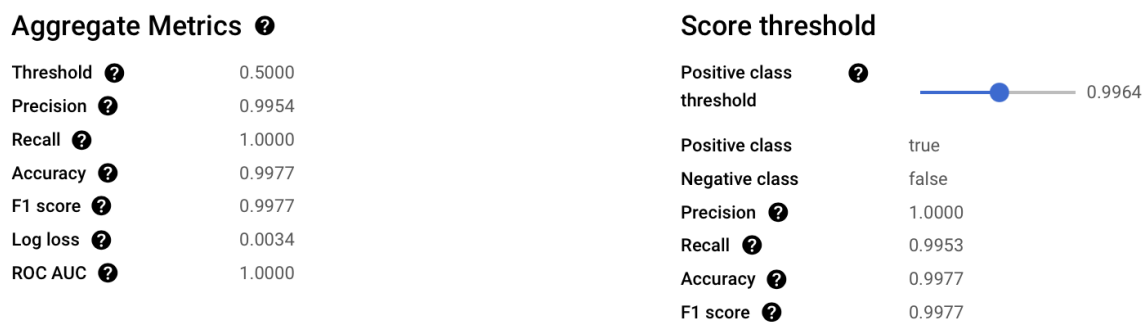
$$precision = (TruePositive) / (TruePositive + FalsePositive) \quad (6.1)$$

$$recall = (TruePositive) / (TruePositive + FalseNegative) \quad (6.2)$$

Figure 6.4 outlines the aggregate metrics for both models and the optimum score thresholds. In part (a), it can be seen that by increasing the threshold, the precision score increases; however, the recall score drops from 99% to 73%. Increasing the threshold score to 0.8845 indicates that the F1 score for the model improves from 68% to 74%. In part (b), the metrics show that model_02 performs well, given the fact that the range of the dataset is smaller (the undersampling method has been incorporated for the training dataset). This means that the model's prediction is reasonably accurate; however, in the later part of this chapter, the new set of test data, including 1093 records that were not included in the training phase, will be examined to determine the accuracy of both models' predictions.



(a) Evaluated Metrics and Optimum Score Threshold (Model_01)



(b) Evaluated Metrics and Optimum Score Threshold (Model_02)

Figure 6.4: Evaluated Metrics - Logistic Regression models (model_01, model_02)

Figure 6.5 corresponds to the selected threshold scores in figure 6.4. As seen in Precision-recall by threshold graph (figure 6.5.a), both lines intersect each other at the positive class threshold of 0.8845 to provide the optimum classification.

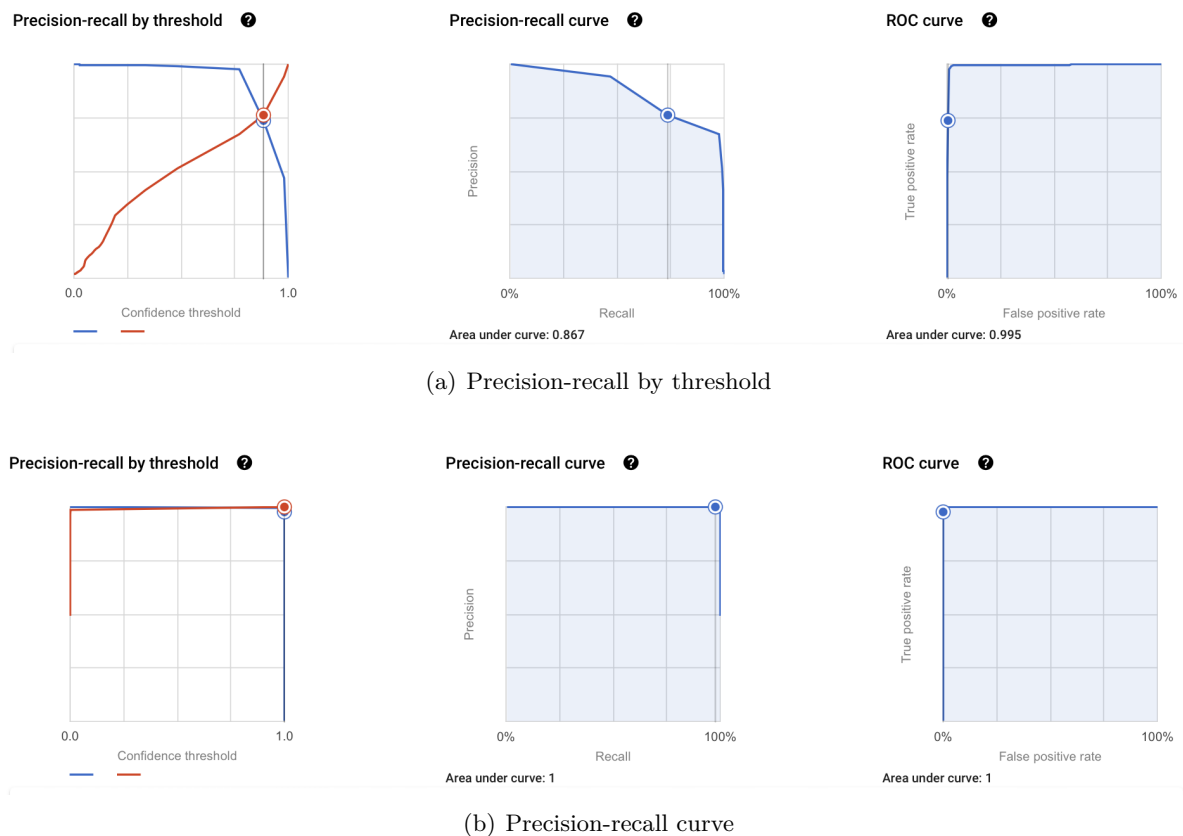


Figure 6.5: Evaluated performance curves - Logistic Regression models (model_01, model_02)

Confusion Matrix

In fraud detection, the predicted score by the algorithm to determine if the transaction is fraudulent or non-fraudulent comes from the binary classification problem. The decision on how the predicted score is to be classified as True or False is based on the threshold value corresponding to the probability of the predicted score. If the probability exceeds the threshold, that transaction classifies as fraud and wise-versa. Table 6.5 illustrates the confusion matrix for both models for the test data that has been examined (the autoencoder model predicted 627 records as anomalous and 466 records as non-anomalous for the total records of 1,093).

		Predicted label	
		False	True
True label (anomalous)	Negative	466	0
	Positive	0	627

(b) confusion matrix of model_01

		Predicted label	
		False	True
True label (anomalous)	Negative	0	466
	Positive	0	627

(b) confusion matrix of model_02

Scenarios	Scenario description	Classification
Scenario 1	The transaction has been detected as anomalous and the model predicted as anomalous	TP
Scenario 2	The transaction has been detected as non-anomalous and the model predicted as anomalous	FP
Scenario 3	The transaction has been detected as non-anomalous and the model predicted as non-anomalous	FN
Scenario 4	The transaction has been detected as anomalous and the model predicted as non-anomalous	TN

Table 6.5: Confusion Matrix (model_01, model_02)

Table 6.5 indicates that for model_01, the prediction was accurate, and for model_02, the anomalous transactions were predicted correctly; however, for non-anomalous transactions, it was predicted with the miss-classified label (FP). This could be due to the limited range of dataset have been used during the training process⁷ (undersampling method was used).

Table 6.6 shows the evaluated scores for the test dataset prediction for both models. Model_02 has a lower precision score, but the true positive transactions have been predicted correctly with the recall score of 100% and 72.90% for the overall f1_score.

Model	Data source	Precision	Recall	Accuracy	f1_score	log_loss	ROC_AUC
Model_01	Dataset_02 (test data)	100%	100%	100%	100%	0.2033	100%
Model_02	Dataset_02 (test data)	57.36%	100%	57.36%	72.90%	3.63	98.91%

Table 6.6: Evaluated metrics - test data records

⁷among 53,295 records, only 2,124 records (1,062 as anomalous and 1,062 as non-anomalous) were selected during the training phase

6.2.3 Time Series Data Models

Time series analyses were conducted for the second part of the anomaly detection analysis. Four models were created for four scenarios based on the ARIMA_PLUS model. Auto Regressive Integrated Moving Average (ARIMA) model can be considered one of the most popular statistical methods for time-series analysis.

The objective was to identify the anomalies based on the total number of transactions received from the sensor devices grouped by the daily and hourly intervals. Two distinct data sets have been used: (1) Active KFA5 sensor devices (Table 5.3), and (2) All KFA5 sensor devices (active and inactive devices).

Figures 6.6, and 6.7 illustrate the workflows of time series analysis for daily and hourly transactions. In hourly transactions, there is an additional step to convert the output values of hours from INT64 to timestamp format. `TIMESTAMP_SECONDS` function is used for this conversion by multiplying with 360 to obtain the timestamp in hours.

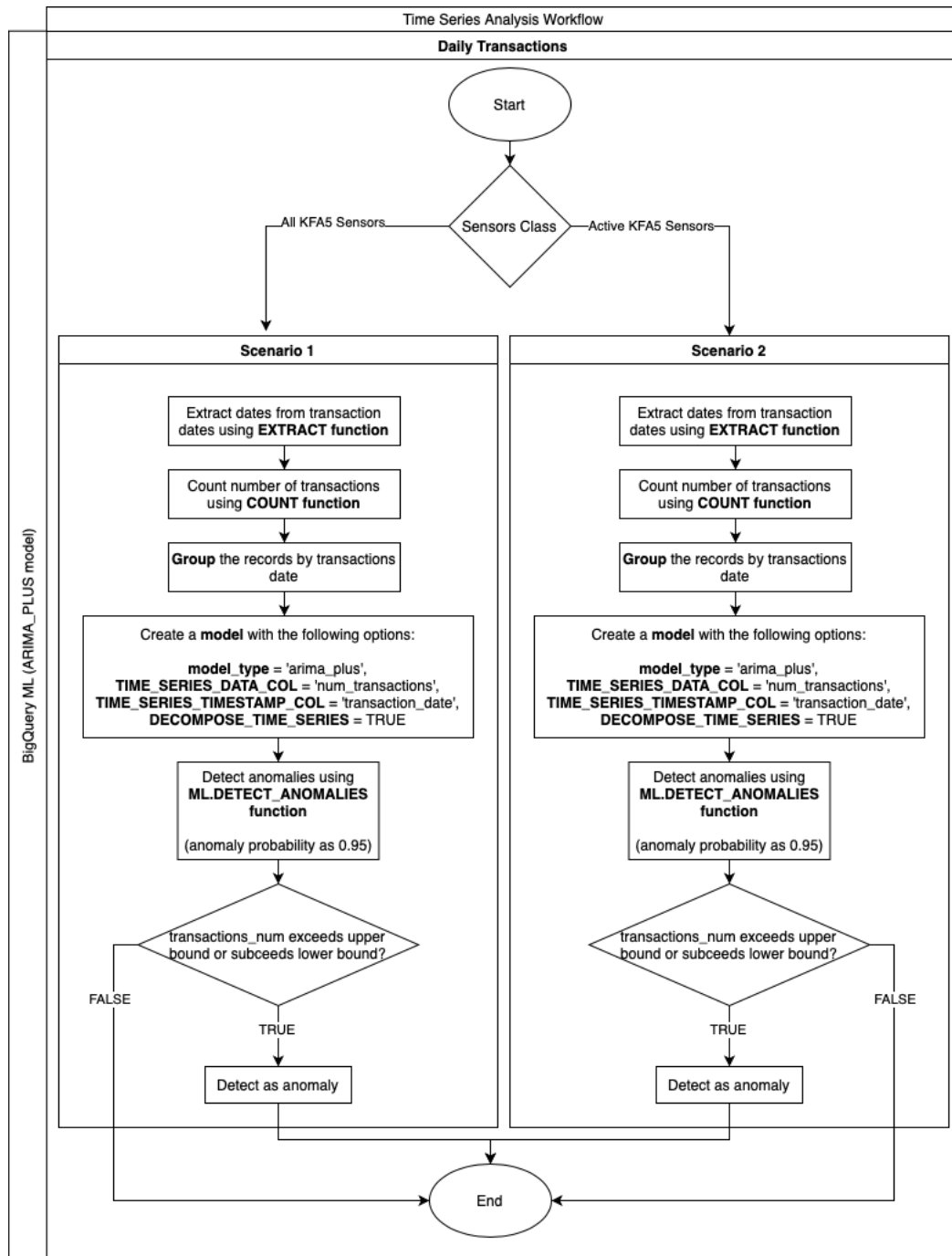


Figure 6.6: Time series analysis workflow (daily transactions) - KFA5 Sensor Devices

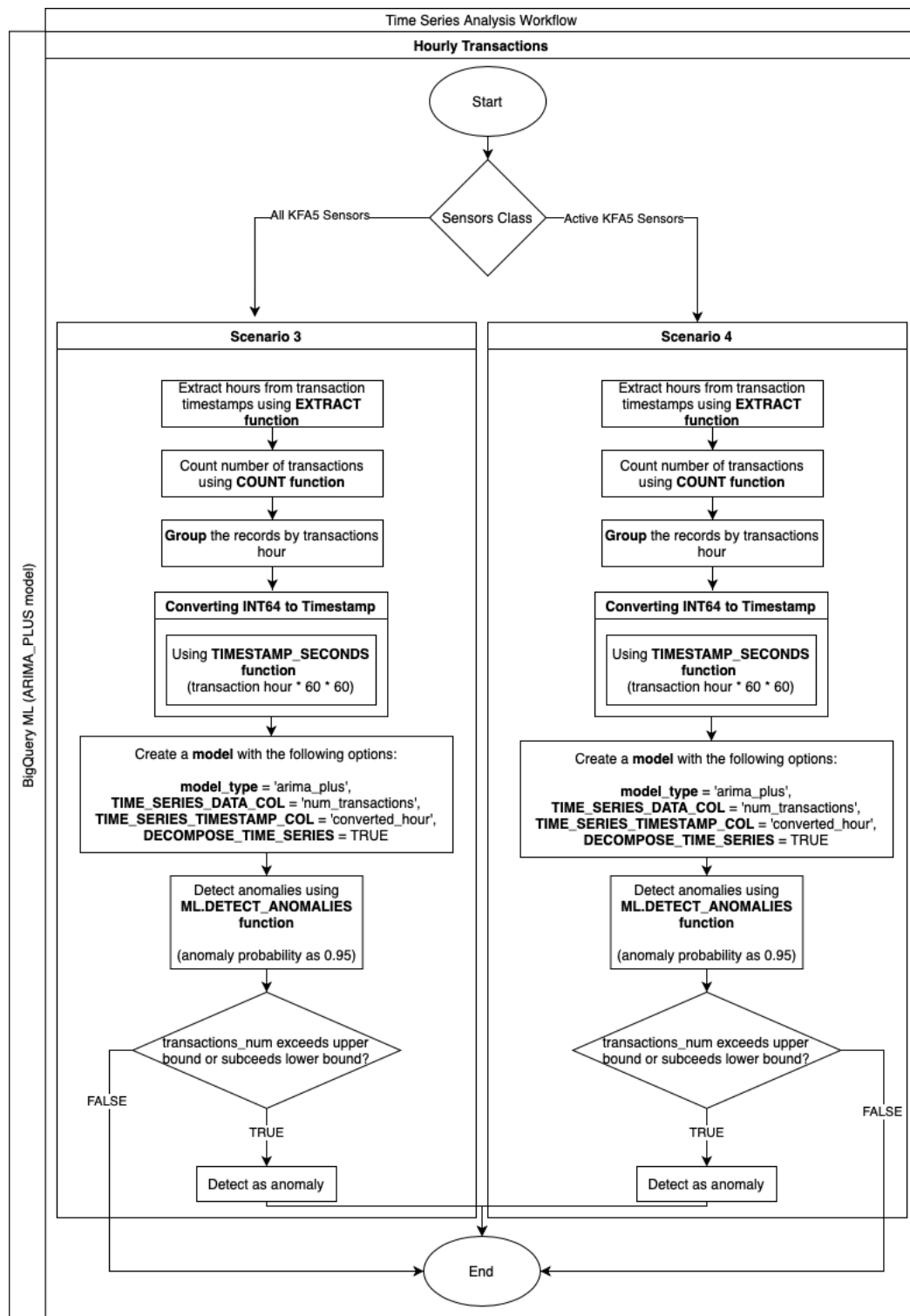


Figure 6.7: Time series analysis workflow (hourly transactions) - KFA5 Sensor Devices

Scenario 1: Anomalies of the total number of daily transactions (KFA5 all sensor devices)

Figure 6.8 illustrates the anomalies detected using the ARIMA_PLUS method followed by the table 6.7 that represents the confidence probability score of the given anomalies (probability percentage). The anomaly probability threshold for this scenario has been set to 95%.

Anomalies for total number of the daily transactions received (KFA5 all sensor devices)

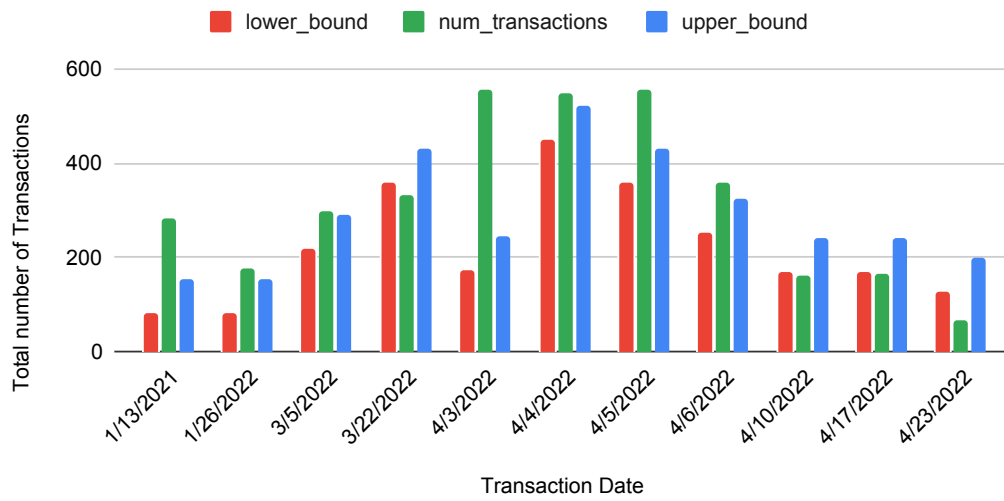


Figure 6.8: Anomalies of total daily transactions (KFA5 All Sensor Devices)

Transaction Date	Lower Bound	Transaction Number	Upper Bound	Anomaly Prob.(%)
1/13/2021	82.4004	282	154.4317	99.99
1/26/2022	82.1280	178	154.1593	99.84
3/5/2022	217.536	297	289.5681	98.16
3/22/2022	357.610	332	429.6412	99.88
4/3/2022	173.018	554	245.0494	100
4/4/2022	450.561	547	522.5929	99.85
4/5/2022	359.672	554	431.7039	99.99
4/6/2022	251.415	360	323.4472	99.97
4/10/2022	167.243	161	239.2747	97.82
4/17/2022	168.799	164	240.8302	97.34
4/23/2022	126.674	68	198.7052	99.99

Table 6.7: Anomalies of total daily transactions (KFA5 All Sensor Devices)

Scenario 2: Anomalies of the total number of daily transactions (KFA5 active sensor devices)

Figure 6.9 illustrates that the number of predicted anomalies are higher in comparison to figure 6.8 with the anomaly probability threshold of 95%.

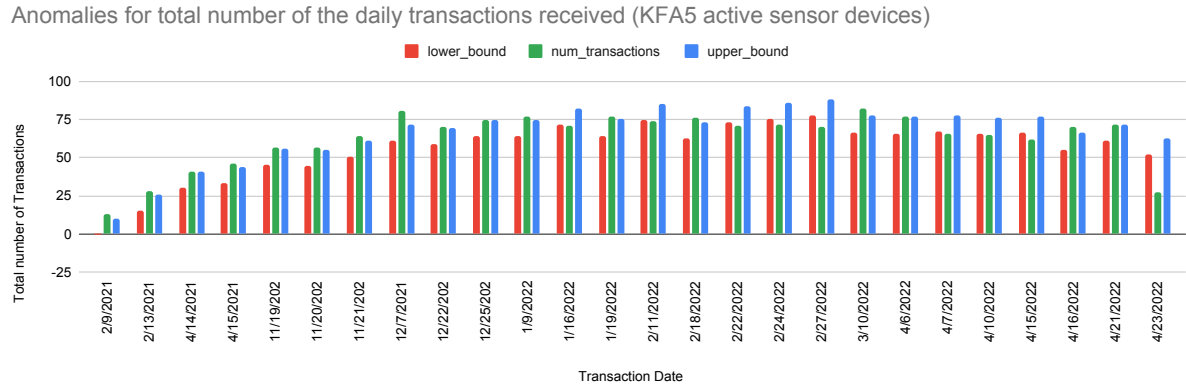


Figure 6.9: Anomalies of total daily transactions (KFA5 Active Sensor Devices)

Transaction Date	Lower Bound	Transaction Number	Upper Bound	Anomaly Prob.(%)
2/9/2021	-0.3891	13	10.2635	99.64
2/13/2021	15.3404	28	25.9931	99.25
4/14/2021	29.9661	41	40.6187	96.42
4/15/2021	33.5794	46	44.2321	99.04
11/19/2021	45.3815	57	56.0342	97.91
11/20/2021	44.5086	57	55.1613	99.11
11/21/2021	50.2965	64	60.9491	99.74
12/7/2021	61.3012	81	71.9539	99.99
12/22/2021	58.9122	70	69.5649	96.59
12/25/2021	64.0861	75	74.7388	96.01
1/9/2022	63.9334	77	74.5861	99.50
1/16/2022	71.5780	71	82.2307	97.00
1/19/2022	64.4741	77	75.1268	99.14
2/11/2022	74.3185	74	84.9712	96.21
2/18/2022	62.6093	76	73.2619	99.64
2/22/2022	73.0050	71	83.6577	99.25
2/24/2022	75.4934	72	86.1460	99.84
2/27/2022	77.7287	70	88.3814	99.99
3/10/2022	66.7053	82	77.3580	99.95
4/6/2022	65.9630	77	76.6157	96.43
4/7/2022	67.2634	66	77.9161	98.43
4/10/2022	65.5462	65	76.1989	96.91
4/15/2022	66.2907	62	76.9433	99.93
4/16/2022	55.4254	70	66.0781	99.89
4/21/2022	61.1168	72	71.7694	95.90
4/23/2022	52.0873	27	62.7400	99.99

Table 6.8: Anomalies of total daily transactions (KFA5 Active Sensor Devices)

Scenario 3: Anomalies of the total number of hourly transactions (KFA5 all sensor devices)

In scenarios 3 and 4, anomalies of hourly transactions are examined. Figure 6.10 illustrates the anomalies detected using the ARIMA_PLUS method followed by the table 6.9 that represents the confidence probability score of the given anomalies (probability percentage). The anomaly probability threshold for this scenario has been set to 95%. The two detected anomalies reflect transaction hours at 7 am and 8 am by exceeding the upper bound limit (highlighted in green).

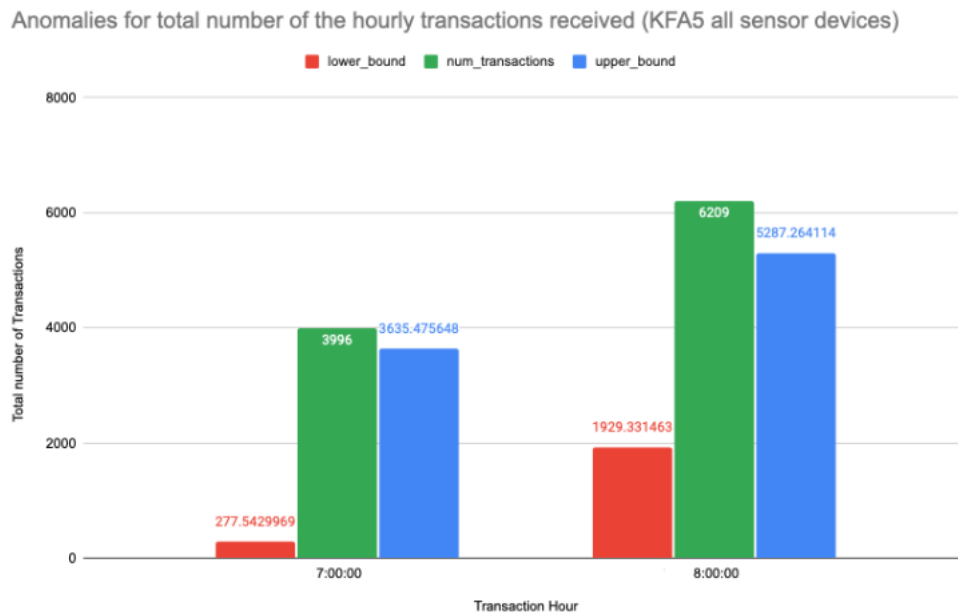


Figure 6.10: Anomalies of total hourly transactions (KFA5 All Sensor Devices)

Transaction Hour	Lower Bound	Transaction Number	Upper Bound	Anomaly Prob.(%)
07:00:00	277.5429	3996	3635.4756	98.24
08:00:00	1929.3314	6209	5287.2641	99.71

Table 6.9: Anomalies of total hourly transactions (KFA5 All Sensor Devices)

Scenario 4: Anomalies of the total number of hourly transactions (KFA5 active sensor devices)

In hourly transactions for KFA5 active sensors, only one anomaly was detected for transaction hour at 8 am. Figure 6.11 shows that the number of transactions for this period exceeded the upper bound limit (highlighted in green).

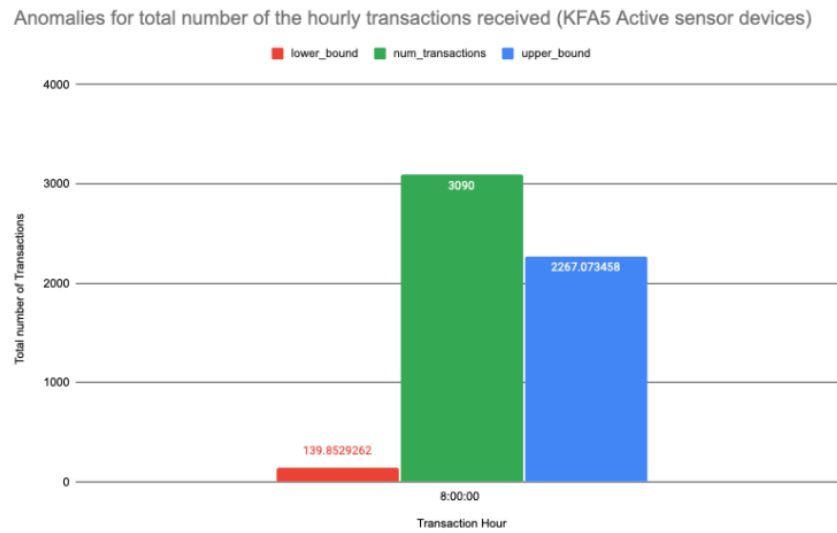


Figure 6.11: Anomalies of total hourly transactions (KFA5 Active Sensor Devices)

Transaction Hour	Lower Bound	Transaction Number	Upper Bound	Anomaly Prob.(%)
08:00:00	139.8529	3090	2267.0734	99.91

Table 6.10: Anomalies of total hourly transactions (KFA5 Active Sensor Devices)

7 Conclusion

In this research study, the author began the study by introducing the WSNs, their importance in today's IoT and computer network industry, and the limitation imposed on embedded systems to provide a secure communication tool to transmit data by adopting advanced cryptographic mechanisms. Then, due to these hardware limitations (computing power resources, energy consumption, and manufacturing costs), cloud computing (in particular, DWaaS) was studied to examine if shifting a load of such security validation and control to the cloud with the help of machine learning techniques can provide the desired level of satisfaction to secure the data transmissions from outsider attacks.

Hence, anomaly detection techniques were incorporated to evaluate if malicious transmitted measured data from the sensor devices can be effectively detected and act as a gateway to protect the data before being stored and processed in the database. For this reason, fraud detection and time series analyses were elected and studied to distinguish normal from abnormal traffics.

Google BigQuery ML platform was chosen to perform this analysis. After the dataset¹ was obtained and compiled, during the first phase of the analysis (fraud detection using the Autoencoder technique), it was observed that the dataset was imbalanced, meaning the proportion of normal and abnormal classes were not equally distributed. This observation imposes a problem for classification techniques in ML anomaly detection whereby since the normal class contains more data during the modeling process, the model is prone to predict anomalies for minority class accurately. Because of this shortcoming, imbalanced learning was used for two respective classification approaches²: For the first classification model, *auto_class_weights* function in BigQuery ML was used, and for the second classification model, the under-sampling method was used. The study has shown that the first classification model with an imbalanced dataset and weighting enabled provides accurate prediction results to detect anomalous transactions compared to the under-sampling technique. As for the second part of the research (time series analysis), four distinctive models were compiled using the ARIMA_PLUS method to detect the total transaction records based on four scenarios: Daily transactions for (1) all (active and non-active), and (2) active KFA5 sensor devices; Hourly transactions for (3) all (active and non-active), and (4) active KFA5 sensor devices. The anomaly probability thresholds for these scenarios were set to be 95%.

The developed pipeline for this research is considered a semi-automated process. Future research could adopt this prototype in Google Vertex AI to obtain a full-fledged automated MLOPS pipeline.

¹the dataset refers to the measured sensor data from various sensor devices

²Logistic Regression method was used

8 Appendix 1: Autoencoder

Autoencoder modeling

```
1  # in this space we try to create a new autoencoder with the optimized feature selection
    and feature transformations based on the classification learner app
2  #features to be used: DeviceID, ULL1, AMPL1, ULL2,AMPL2,ULL3,AMPL3,SIGMIN, SIGMAX,
    SIGSTD, ActiveSensorClass, Batt,, CorrMess, FM, ProductID, TankID,Temperature_C,
    Temperature_F
3
4  # Creating a new autoencoder model (V4)
5  CREATE MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    autoencoder_model_V4_SelectedFeatures`
6  OPTIONS(
7    model_type='autoencoder',
8    activation_fn='relu', #RELU refers to REctified linear
9    batch_size=8,
10   dropout=0.2,
11   hidden_units=[32,16,4,16,32],
12   learn_rate=0.001,
13   l1_reg_activation=0.0001,
14   max_iterations=10,
15   optimizer='adam'
16 ) AS
17 SELECT * EXCEPT(PrimaryKey,DateTime,ProductID,weight_col)
18 FROM
19   `precise-ray-353609.20220715_KFA5_Compiled.table01`
20
21 # to predict the anomalies
22 SELECT *
23 FROM
24   ML.DETECT_ANOMALIES(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    autoencoder_model_V4_SelectedFeatures`,
25   STRUCT(0.02 AS contamination),
26   TABLE `precise-ray-353609.20220715_KFA5_Compiled.table01`);
27
```


9 Appendix 2: Imbalanced learning

This section covers the undersampling technique which is used for logistic regression modeling (imbalanced learning)

Undersampling query

```
1  # There are 1062 anomalies detected by the autoencoder out of 53,295 transactions
2  # approach 1. Undersampling
3  # based on the minority class, 1062 non-anomalous transactions are collected
4  # (from majority class) and combined with. Overall, 2124 transaction will be used.
5
6  (SELECT * FROM `precise-ray-353609.05_model_output.autoencoder_V4_Results_all`
7   WHERE is_anomaly = FALSE
8   LIMIT 1062)
9  UNION ALL
10 (SELECT * FROM `precise-ray-353609.05_model_output.autoencoder_V4_Results_all`
11  WHERE is_anomaly = TRUE
12  LIMIT 1062)
13
```


10 Appendix 3: Logistic Regression

The code in appendix-2 covers logistic regression (binary) classification learning. There are two approaches for logistic regression learning: (1) using auto weight function for the imbalanced dataset, and (2) using undersampling technique for the balanced dataset.

Option 1: Auto weighting using imbalanced dataset

```
1  ## Creating a LOGISTIC REGRESSOIN Model to predict the anomalies based on the
   autoencoder predicted outputs (response class)
2  # Option 1: Using Weighting to avoid for overfitting for imbalanced data
3  # functional_weight allocates 80% of the data for training, and reserves the remaining
   for evaluation and predictions
4  # the selected features are shown as below
5  # note: here we are calling collected the reconstructed losses data from autoencoder V4
6  SELECT * EXCEPT(mean_absolute_error,mean_squared_error,mean_squared_log_error,PrimaryKey,
   DateTime,ProductID,weight_col)
7  FROM `05_model_output.autoencoder_model_V4_results_RECONSTRUCTION_LOSS_AllFeatures`
8
9  # First we create a view in BigQuery ML, that helps to observe the data before executing
   for model creation.
10 CREATE OR REPLACE VIEW
11 | `precise-ray-353609.20220715_KFA5_Compiled.V4AutoencoderResults_BQML_LOGREG_view` AS
12 SELECT
13 | * EXCEPT(mean_absolute_error,mean_squared_error,mean_squared_log_error,PrimaryKey,
   DateTime,ProductID,weight_col)
14 FROM `05_model_output.autoencoder_model_V4_results_RECONSTRUCTION_LOSS_AllFeatures`
15 |
16 # the auto_class_weights is the function built in BigQuery ML where automatically
   balance the imblanace dataset for classification models (e.g., Logistic Regression)
17 # input_label_col is the response (target) label generated from the Autoencoder model
18 CREATE OR REPLACE MODEL
19 | `precise-ray-353609.20220715_KFA5_Compiled.V4AutoencoderResults_BQML_LOGREG_model_v1`
20 OPTIONS
21 | ( model_type = 'LOGISTIC_REG',
   |   auto_class_weights=TRUE,
   |   input_label_cols = ['is_anomaly']
22 | ) AS
23
24
25 SELECT
26 | * EXCEPT(mean_absolute_error,mean_squared_error,mean_squared_log_error,PrimaryKey,
   DateTime,ProductID,weight_col)
27 FROM `05_model_output.autoencoder_model_V4_results_RECONSTRUCTION_LOSS_AllFeatures`
28
```

```
29 # Once the first model is created, we can obtain the evaluated results for trained
    # dataset, then the test data will be examined
30 SELECT *
31 FROM ML.EVALUATE(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_model_v1`,
32 (
33 | SELECT * FROM `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_view`
34 )
35 )
36
37 # ML.EXPLAIN_PREDICT is a function in BigQuery ML to provide further information about
    # the predicted query
38 SELECT *
39 FROM ML.EXPLAIN_PREDICT(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_model_v1`,
40 (
41 | SELECT *
42 | FROM `precise-ray-353609.20220715_KFA5_Compiled.V4AutoencoderResults_BQML_LOGREG_view`
43 |), STRUCT (5 as top_k_features)
44 )
45
46 # ML.GLOBAL_EXPLAIN is another function to provide information about the model
47 SELECT *
48 FROM ML.GLOBAL_EXPLAIN(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_model_v1`)
49
50
```

Option 2: Undersampling technique for the balanced dataset

```

69
70 # Option 2: Using imbalanced learning (undersampling approach)
71 # now we create new version using the balanced dataset
72 # list of features selected: is_anomaly, TankID, DeviceID, Temp_C, Temp_F, ULL1, AMPL1,
    ULL2, AMPL2, ULL3, AMPL3, SIGMIN, SIGMAX, SIGSTD, FM, Batt, Crssi, CorrMess,
    ActiveSensorClass
73
74 CREATE OR REPLACE VIEW
75 | `precise-ray-353609.20220715_KFA5_Compiled.V4AutoencoderResults_BQML_LOGREG_v2_view` AS
76 SELECT * EXCEPT(mean_squared_error, PrimaryKey, ProductID, DateTime)
77 FROM `precise-ray-353609.05_model_output.ImbalancedDataset_Undersampling_v1`
78
79 # =====
80 # creating a model using undersampling dataset
81 CREATE OR REPLACE MODEL
82 | `precise-ray-353609.20220715_KFA5_Compiled.V4AutoencoderResults_BQML_LOGREG_model_v2`
83 OPTIONS
84 | ( model_type = 'LOGISTIC_REG',
85 |   auto_class_weights=TRUE,
86 |   input_label_cols = ['is_anomaly']
87 | ) AS
88 SELECT * EXCEPT(mean_squared_error, PrimaryKey, ProductID, DateTime)
89 FROM `precise-ray-353609.05_model_output.ImbalancedDataset_Undersampling_v1`
90
91 # evaluate the model using ML.EVALUATE function
92 SELECT *
93 FROM ML.EVALUATE(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_model_v2`,
94 (
95 | SELECT * FROM `precise-ray-353609.20220715_KFA5_Compiled.
    V4AutoencoderResults_BQML_LOGREG_v2_view`
96 | )
97 )
98

```


11 Appendix 4: Models evaluations

Comparing distinct dataset for test data

```
1  # comparing different datasets and select the distinct rows
2  (
3      SELECT * FROM `precise-ray-353609.KFA5_EU.Heating_Oil_Dataset01`
4      EXCEPT DISTINCT
5      SELECT * FROM `precise-ray-353609.KFA5_EU.Used_Oil_Dataset01`
6      EXCEPT DISTINCT
7      SELECT * FROM `precise-ray-353609.KFA5_EU.Used_Oil_Dataset02`
8  )
```

Reconstruction loss query

```
1  # to generate the reconstruction loss for autoencoder model (V4)
2  SELECT *
3  FROM
4  ML.RECONSTRUCTION_LOSS(MODEL `20220715_KFA5_Compiled.
5  autoencoder_model_V4_AllFeatures_ExceptDatetime`,
6  TABLE `05_model_output.autoencoder_V4_Results_excluding_DateTime_TESTDATA_V2`
7  )
```

Models evaluation queries (LOGREG)

```

1  # models
2  # model 1 `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v1`
3  # model 2 `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v2`
4  # test data `precise-ray-353609.KFA5_EU.TEST_DATA_V2`
5
6  # LOGREG_model01
7  SELECT *
8  FROM
9  | ML.CONFUSION_MATRIX(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v1`,
10 | (SELECT * EXCEPT(PrimaryKey, ProductID,DateTime, mean_squared_error)
11 |   FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))
12
13 SELECT * FROM
14 | ML.ROC_CURVE(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v1`,
15 | (SELECT * EXCEPT(PrimaryKey, ProductID,DateTime, mean_squared_error)
16 |   FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))
17
18 # LOGREG_model02
19 SELECT *
20 FROM
21 | ML.CONFUSION_MATRIX(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v2`,
22 | (SELECT * EXCEPT(PrimaryKey, ProductID,DateTime, mean_squared_error)
23 |   FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))
24
25 SELECT * FROM
26 | ML.ROC_CURVE(MODEL `precise-ray-353609.20220715_KFA5_Compiled.
   V4AutoencoderResults_BQML_LOGREG_model_v2`,
27 | (SELECT * EXCEPT(PrimaryKey, ProductID,DateTime, mean_squared_error)
28 |   FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))

```



```

# precision-recall curve
SELECT recall, true_positives / (true_positives + false_positives) AS precision
FROM ML.ROC_CURVE(MODEL `20220715_KFA5_Compiled.
V4AutoencoderResults_BQML_LOGREG_model_v1`,
(SELECT * FROM `05_model_output.autoencoder_V4_Results_all`))

SELECT recall, true_positives / (true_positives + false_positives) AS precision
FROM ML.ROC_CURVE(MODEL `20220715_KFA5_Compiled.
V4AutoencoderResults_BQML_LOGREG_model_v1`,
(SELECT * FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))

```

```
# precision-recall curve
SELECT recall, true_positives / (true_positives + false_positives) AS precision
FROM ML.ROC_CURVE(MODEL `20220715_KFA5_Compiled.
V4AutoencoderResults_BQML_LOGREG_model_v2`,
(SELECT * FROM `05_model_output.autoencoder_V4_Results_all`))

SELECT recall, true_positives / (true_positives + false_positives) AS precision
FROM ML.ROC_CURVE(MODEL `20220715_KFA5_Compiled.
V4AutoencoderResults_BQML_LOGREG_model_v2`,
(SELECT * FROM `05_model_output.autoencoder_V4_Results_all_TESTDATA_V2`))
```


12 Appendix 5: Time Series Analysis

Daily transaction (all sensors)

```
1 CREATE OR REPLACE MODEL
2 `precise-ray-353609.20220715_KFA5_Compiled.transactions_daily`
3 OPTIONS(
4   model_type='arima_plus',
5   TIME_SERIES_DATA_COL='num_transactions',
6   TIME_SERIES_TIMESTAMP_COL='transaction_date',
7   DECOMPOSE_TIME_SERIES=TRUE
8 )
9 AS (
10   SELECT
11     EXTRACT(date from DateTime) AS transaction_date,
12     COUNT(*) AS num_transactions
13   FROM `precise-ray-353609.20220715_KFA5_Compiled.table01`
14   GROUP BY transaction_date
15 );
```

Daily transaction (active sensors)

```
1 CREATE OR REPLACE MODEL
2 `precise-ray-353609.20220715_KFA5_Compiled.transactions_daily_ActiveSensors`
3 OPTIONS(
4   model_type='arima_plus',
5   TIME_SERIES_DATA_COL='num_transactions',
6   TIME_SERIES_TIMESTAMP_COL='transaction_date',
7   DECOMPOSE_TIME_SERIES=TRUE
8 )
9 AS (
10   SELECT
11     EXTRACT(date from DateTime) AS transaction_date,
12     COUNT(*) AS num_transactions
13   FROM `precise-ray-353609.20220715_KFA5_Compiled.table01`
14   WHERE ActiveSensorClass = 1
15   GROUP BY transaction_date
16 );
```

Anomalies of the total number of hourly transactions

The code covers both KFA5 active and non-active sensor devices

```
1 # this space covers the time-series analysis for anomalies related to number of
  transactions per hour
2 # first following query obtain the total number of transactions per hour
3 SELECT
4     EXTRACT(hour from DateTime) AS transaction_hour,
5     COUNT(*) AS num_transactions
6 FROM `precise-ray-353609.20220715_KFA5_Compiled.table01`
7 GROUP BY transaction_hour
8
9 # save the table as `transactions_hourly` containing two columns: (1) hour, (2)
  num_transactions
10
11 # next, following query convert the hour values from INT64 to Timestamp
12
13 SELECT TIMESTAMP_SECONDS(hour*60*60) as converted_hour, num_transactions
14 FROM `05_model_output.hourly_transactions`
15
16 # model creation
17 CREATE OR REPLACE MODEL
18     `precise-ray-353609.20220715_KFA5_Compiled.transactions_hourly`
19 OPTIONS(
20     model_type='arima_plus',
21     TIME_SERIES_DATA_COL='num_transactions',
22     TIME_SERIES_TIMESTAMP_COL='converted_hour',
23     DECOMPOSE_TIME_SERIES=TRUE
24 )
25 AS (
26     SELECT
27         TIMESTAMP_SECONDS(hour*60*60) as converted_hour, num_transactions
28     FROM `05_model_output.hourly_transactions`
29 );
30 |
31 # detecting anomalies
32 SELECT *
33 FROM ML.DETECT_ANOMALIES(
34     MODEL `precise-ray-353609.20220715_KFA5_Compiled.transactions_hourly`,
35     STRUCT (0.95 AS anomaly_prob_threshold)
36 ) ORDER BY anomaly_probability DESC
```

Abbreviations and Acronyms

List of abbreviations and acrynyns used in this report can be found in Table 12.1.

Abbrv	Full Terminology
ANN	Artificial neural network
ARIMA	Auto Regressive Integrated Moving Average
MAE	Mean-absolute-error
MSE	Mean-squared-error
MSLE	Mean-squared-log-error
PR curve	Precision-recall curve
API	Application Programming Interface
AD	Anomaly Detection
BTU	Brandenburg University of Technology
DWaaS	Data Warehouse as a Service
GUI	Graphical User Interface
ocSVM	One-Class Support Vector Machine
LOF	Local Outlier Factor
kNN	k-Nearest Neighbor
LOGREG	Logistic Regression
IoT	Internet of Things
ML	Machine Learning
DoS attack	Denial of service attack
DDoS attack	Distributed denial of service attack
ULL	Ullage
PR curve	precision-recall curve
MATLAB	MATrix LABoratory
MLOPS	Machine Learning Operations
WSN	Wireless Sensor Network

Table 12.1: Abbreviations and Acronym

Bibliography

- [1] .
- [2] M. G. R. C. P. B. K. F. H. Alberto Fernández Salvador García. *Learning from Imbalanced Data Sets*. Springer, 2018.
- [3] *Anomaly Fraud detection*. <https://towardsdatascience.com/anomaly-fraud-detection-a-quick-overview-28641ec49ec1>, author=Alireza Hashemi, note = Accessed: 2022-10-11.
- [4] *Anomaly Detection with Autoencoders Made Easy*. <https://towardsdatascience.com/anomaly-detection-with-autoencoder-b4cdce4866a6>, year=October 1, 2021 note = Accessed: 2022-09-07.
- [5] *ARIMA_PLUS*. <https://cloud.google.com/bigquery-ml/docs/reference/standard-sql/bigqueryml-syntax-create-time-series>, note = Accessed: 2022-08-28.
- [6] B. Berisha, E. Mëziu and I. Shabani. *Big data analytics in Cloud computing: an overview*. In: *Journal of Cloud Computing* 11 (2022), p. 24.
- [7] *BigQuery, a serverless data warehouse*. <https://medium.com/sajari/bigquery-a-serverless-data-warehouse-9a4d623f3f43>, author=Hamish Ogilvy, note = Accessed: 2022-10-25.
- [8] N. R. S. J. Breunig M.M. Kriegel H.-P. Lof: *Identifying density-based local outliers*. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, Association for Computing Machinery (2000)* Vol. 29 (), pp. 93–104.
- [9] M. M. Breunig, H. P. Kriegel et al. *LOF: Identifying density-based local outliers*. In: *SIGMOD Record (ACM Special Interest Group on Management of Data)* 29 (2 2000).
- [10] A. Canziani. *Introduction to autoencoders · Deep Learning*. <https://atcold.github.io/pytorch-Deep-Learning/en/week07/07-3>, note = Accessed: 2022-08-21.
- [11] V. Chandola, A. Banerjee and V. Kumar. *Anomaly Detection*. In: *Encyclopedia of Machine Learning and Data Mining*. Ed. by C. Sammut and G. I. Webb. Boston, MA: Springer US, 2016, pp. 1–15.
- [12] *Classification with Imbalanced Data*. <https://de.mathworks.com/help/stats/classification-with-imbalanced-data.html>, note = Accessed: 2022-09-16.
- [13] B. I. David C. Hoaglin. *Volume 16: How to Detect and Handle Outliers*. In: 2013.
- [14] D. Giusto, A. Iera et al. *The Internet of Things: 20th Tyrrhenian Workshop on Digital Communications*. 2010.
- [15] R. Hawley and N. Gallagher. *On Edgeworth’s method for minimum absolute error linear regression*. In: *IEEE Transactions on Signal Processing* 42.8 (1994), pp. 2045–2054.
- [16] *Independent and identically distributed random variables*. https://en.m.wikipedia.org/wiki/Independent_and_identically_distributed_random_variables. Accessed October 2022. 2022.

- [17] *Intro to Autoencoders*. <https://towardsdatascience.com/introduction-to-arima-for-time-series-forecasting-ee0bc285807a>, note = Accessed: 2022-08-28.
- [18] H. Kaur, P. Agrawal and A. Dhiman. *Visualizing Clouds on Different Stages of DWH - An Introduction to Data Warehouse as a Service*. In: *2012 International Conference on Computing Sciences*. 2012, pp. 356–359.
- [19] *KNN*. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* ().
- [20] P. Kocher, R. Lee et al. *Security as a new dimension in embedded system design*. In: 2004.
- [21] S. P. T. Krishnan and J. L. U. Gonzalez. *Google BigQuery*. In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects*. Berkeley, CA: Apress, 2015, pp. 235–253.
- [22] T. Kyung. *3 Types of Anomalies in Anomaly Detection*. <https://hackernoon.com/3-types-of-anomalies-in-anomaly-detection>, note = Accessed: 2022-08-28.
- [23] T. Kyung. *Anomaly Detection for Industrial Processes and Machinery with MATLAB*. https://de.mathworks.com/videos/anomaly-detection-for-industrial-processes-and-machinery-with-matlab-1648135588169.html?s_tid=srchtitle_anomaly%20detection_2, note = Accessed: 2022-08-26.
- [24] LaprinthX. *Better, faster, smarter: ELT vs. ETL*. <https://www.matillion.com/resources/blog/better-faster-smarter-elt-vs-etl>, note = Accessed: 2022-10-27,
- [25] Y.-A. Le Borgne, W. Siblini et al. *Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook*. Université Libre de Bruxelles, 2022.
- [26] F. T. Liu, K. M. Ting and Z. H. Zhou. *Isolation forest*. In: 2008.
- [27] *Local outlier factor*. https://en.wikipedia.org/wiki/Local_outlier_factor?ref=hackernoon.com, note = Accessed: 2022-08-28.
- [28] K. M.A. *Nonlinear principal component analysis using autoassociative neural networks*. In: *AIChE J.* 37 (2) (1991), pp. 233–243.
- [29] P. Marwedel. *Embedded Design System: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*. 4th ed. Springer Cham, 2021.
- [30] *Mean squared logarithmic error (MSLE)*. [https://peltarion.com/knowledge-center/modeling-view/build-an-ai-model/loss-functions/mean-squared-logarithmic-error-\(msle\)](https://peltarion.com/knowledge-center/modeling-view/build-an-ai-model/loss-functions/mean-squared-logarithmic-error-(msle)), note = Accessed: 2022-09-16.
- [31] J. L. Polong Lin. *What’s new with BigQuery ML: Unsupervised anomaly detection for time series and non-time series data*. <https://cloud.google.com/blog/products/data-analytics/bigquery-ml-unsupervised-anomaly-detection>, year=July 1, 2021, note = Accessed: 2022-08-28.
- [32] *Proteus __ Level sensors and robotics*. <https://www.proteus-sensor.de>, note = Accessed: 2022-09-12.
- [33] *Proteus EcoFrog WLAN*. <https://www.proteus-sensor.de/shop/Proteus-EcoFrog-WLAN-p241593813>. Accessed: 2022-02-15.
- [34] D. Samariya and A. Thakkar. *A Comprehensive Survey of Anomaly Detection Algorithms*. In: *Annals of Data Science* (2021).
- [35] B. Schölkopf, R. Williamson et al. *Support vector method for novelty detection*. In: 2000.

-
- [36] S. A. S.-T. J. P. J. Schölkopf B. Williamson R. *Support vector method for novelty detection*. In: *Proceedings of the 12th International Conference on Neural Information Processing Systems, NIPS'99, MIT Press, Cambridge, MA, USA (1999)* (), pp. 582–588.
 - [37] *SMOTE: Synthetic Minority Over-sampling Technique*. <https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume16/chawla02a-html/chawla2002.html>, author=Nitesh V. Chawla1, Kevin W. Bowyer2, Lawrence O. Hall1, W. Philip Kegelmeyer, publisher=Journal of Artificial Intelligence Research, year=2002.
 - [38] D. M. Tax and R. P. Duin. *Support Vector Data Description*. In: *Machine Learning* 54 (1 2004).
 - [39] *The ML.RECONSTRUCTION function*. <https://cloud.google.com/bigquery-ml/docs/reference/standard-sql/bigqueryml-syntax-reconstruction-loss>, year=2022-09-14, note = Accessed: 2022-08-28.
 - [40] *Types of Ensemble methods in Machine learning*. <https://towardsdatascience.com/types-of-ensemble-methods-in-machine-learning-4ddaf73879db>, author=Anju Rajbangshi, note = Accessed: 2022-10-10.
 - [41] *Using Connected Sheets*. <https://cloud.google.com/bigquery/docs/connected-sheets>, note = Accessed: 2022-08-28.
 - [42] M. E. Villa-Pérez, M. Álvarez Carmona et al. *Semi-supervised anomaly detection algorithms: A comparative summary and future research directions*. In: *Knowledge-Based Systems* 218 (2021), p. 106878.
 - [43] *What is BigQuery ML*. <https://cloud.google.com/bigquery-ml/docs/introduction>. Accessed October 2022. 2022.
 - [44] Y. Zhao, Z. Nasrullah and Z. Li. *PyOD: A python toolbox for scalable outlier detection*. In: *Journal of Machine Learning Research* 20 (2019).